MODULE 6

Streams and Functional Programming

Leverage Java Streams and functional programming to write concise, expressive, and efficient code.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

STREAMS AND FUNCTIONAL PROGRAMMING

Declarative, Functional Style for Processing Collections

1. STREAM PIPELINE

DATA SOURCE

Collections, Arrays, I/O channels, Generators

```
List<String> names =
List.of("Ana","Bob",
"Cara","Dan","Eve");
Stream<String> stream =
names.stream();
```

INTERMEDIATE OPERATIONS

- **filter()** Predicate<T> — Filters elements based on condition
- **map()** Function<T,R> — Transforms each element
- **sorted()** Comparator<T> — Sorts elements
- ... — Can be chained

TERMINAL OPERATIONS

- **forEach()** Consumer<T> — Perform action on each element
- **collect()** Collector<T,A,R> — Collect results into collection
- **reduce()** BinaryOperator<T> — Combine elements into single result

RESULT

Result produced by terminal operation

2. FUNCTIONAL INTERFACES (CORE BUILDING BLOCKS)

Predicate<T>	Function<T,R>	Consumer<T>	Supplier<T>	BinaryOperator<T>
T -> boolean	T -> R	T -> void	() -> T	(T,T) -> T
Test a condition	Transform input to output	Perform action, no return	Supply / provide a value	Combine two values into one

3. EXAMPLE: PROCESSING WITH STREAMS

INPUT

List<Integer> numbers

```
[ 5, 2, 9, 1, 5, 6 ]
```

```
int sumOfSquares = numbers.stream()
    .filter(n -> n > 2)          // 5, 9, 5, 6
    .map(n -> n * n)             // 25, 81, 25, 36
    .distinct()                  // 25, 81, 36
    .sorted()                    // 25, 36, 81
    .reduce(0, Integer::sum);    // 142
```

OUTPUT

142

(Sum of squares)

4. IMPERATIVE vs FUNCTIONAL PARADIGM

IMPERATIVE STYLE (How to do it)

```
int sum = 0;
for (int n : numbers) {
   if (n > 2) {
      int sq = n * n;
      sum += sq;
   }
}
```

VS

FUNCTIONAL STYLE (What to do)

```
int sum = numbers.stream()
     .filter(n -> n > 2)
     .map(n -> n * n)
     .reduce(0, Integer::sum);
```

Functional style is more concise, readable, and easy to reason about. Focus on WHAT should be done, not HOW.

5. KEY CHARACTERISTICS

- **Declarative** — Express logic without controlling flow
- **Composable** — Operations can be chained seamlessly
- **Lazy Evaluation** — Intermediate operations are executed only when needed
- **Efficient** — Optimized for performance, can leverage parallelism

6. BENEFITS

- **Concise & Readable** — Less boilerplate code, intent is clear
- **Easier Maintenance** — Modular and composable operations
- **Parallal Processing** — Easy to parallelize with `parallelStream()`
- **Better Performance** — Internal optimizations by Stream API

NOTE: Streams do not change the original data source. They provide a functional view for processing data.

Common Data Sources: Collection, Array, Files, Generate, Iterate

MODULE OVERVIEW

Leverage Java Streams and functional programming features to write concise, expressive, and efficient code.

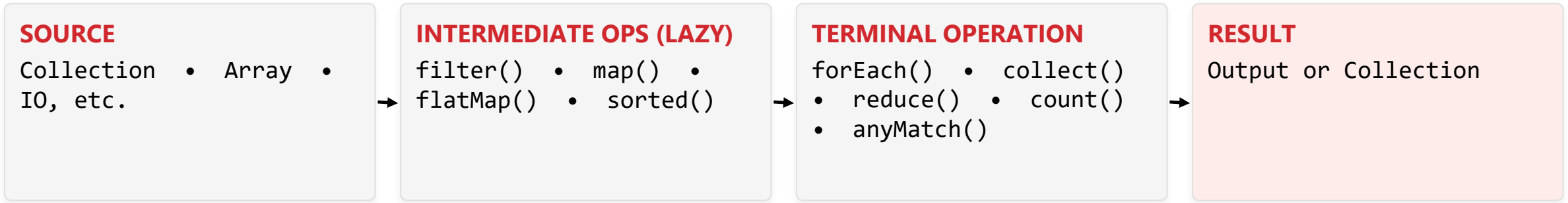
WHAT YOU WILL LEARN

- Introduction to Streams and the Stream Pipeline
- Creating Streams from Collections, Arrays, and other sources
- Intermediate Operations: filter, map, flatMap, sorted, distinct, peek, limit, skip
- Terminal Operations: forEach, collect, reduce, count, anyMatch, allMatch, noneMatch, findFirst
- Stream Pipeline Concepts: lazy evaluation, short-circuiting, stateless vs stateful operations
- Functional Interfaces: Predicate, Function, Consumer, Supplier
- Lambda Expressions and Method References
- Optional class and handling absence of values
- Best Practices and Performance Considerations

KEY TAKEAWAY Streams + Functional Programming enable declarative, readable, and efficient data processing while reducing boilerplate.

THE STREAM PIPELINE

Every stream operation in this module follows the same four-stage shape.



- Source: where the data comes from — Collection, Array, IO, etc.
- Intermediate operations transform the stream and are lazily evaluated.
- The terminal operation triggers execution and produces the result.

KEY TAKEAWAY Keep this four-stage picture in mind — every topic in this module fits somewhere on this pipeline.

CORE FEATURES & COMMON USE CASES

What makes Streams different, and where you'll reach for them.

Declarative Style

Focus on what to do, not how to do it.

Concise & Expressive

Less code, more clarity.

Functional Paradigm

Immutability, pure functions, no side effects.

Performance Optimized

Lazy evaluation, short-circuiting improve efficiency.

Parallel Ready

Easy to parallelize with parallel() for large datasets.

- Common use cases: data filtering & transformation, aggregations & reductions,
- data analysis & reporting, search & matching, pipeline data processing.

KEY TAKEAWAY Streams shine anywhere you're transforming, filtering, or summarizing a collection of data.

WHY FUNCTIONAL PROGRAMMING MATTERS

Functional programming helps us write code that is cleaner, safer, easier to test, and built for modern, scalable applications.

Cleaner & More Concise

- Express complex logic in fewer lines.

Less Error-Prone

- Immutable data and pure functions reduce bugs.

Easier to Test

- Pure functions are predictable, simple to unit test.

Better for Parallelism

- Stateless operations can run in parallel.

Designed for Modern Systems

- Aligns with streams, reactive systems, cloud apps.

Improves Productivity

- Focus on what to do, not how to do it.

KEY TAKEAWAY Six reasons, one theme: functional style reduces the surface area for bugs.

FUNCTIONAL PROGRAMMING IN JAVA

Treat computation as the evaluation of mathematical functions

CORE PRINCIPLES

FUNCTIONS AS FIRST-CLASS CITIZENS
Functions can be assigned, passed and returned.

IMMUTABILITY
Prefer immutable data and avoid changing state.

NO SIDE EFFECTS
Functions should not modify external state; pure functions.

HIGHER-ORDER FUNCTIONS
Functions that take other functions as arguments or return them.

FUNCTION COMPOSITION
Build complex operations by composing simple functions.

ENABLING FEATURES IN JAVA

- Lambda Expressions
- Functional Interfaces
- Stream API
- Optional Class
- Method References

FROM IMPERATIVE TO FUNCTIONAL STYLE

Imperative Approach (How)

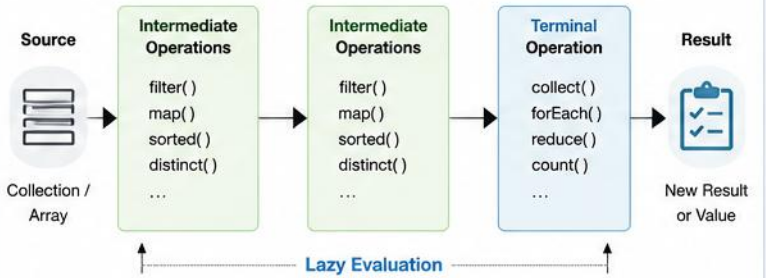
```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> evenSquares = new ArrayList<>();
for (int n : numbers) {
    if (n % 2 == 0) {
        evenSquares.add(n * n);
    }
}
```

Functional Approach (What)

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> evenSquares = numbers.stream()
            .filter(n -> n % 2 == 0)
            .map(n -> n * n)
            .collect(Collectors.toList());
```

Focus on what to do, not how to do it.

STREAM PIPELINE WORKFLOW



BENEFITS OF FUNCTIONAL PROGRAMMING

CONCISENESS
Less boilerplate code, more expressive.

BETTER READABILITY
Code intent is clear and easier to understand.

EASY TESTING
Pure functions are predictable and easy to test.

PARALLELISM READY
Immutable, stateless code works well with parallel streams.

COMPOSABILITY
Small functions can be combined to build complex logic.

FUNCTIONAL INTERFACE EXAMPLES

Interface	Method Signature	Example Use
Predicate<T>	boolean test(T t)	Filtering (filter) Validation
Function<T,R>	R apply(T t)	Mapping (map) Transformation
Consumer<T>	void accept(T t)	Processing (forEach) Side-effect operations
Supplier<T>	T get()	Providing values (Supply)

Lambda Examples

```
x -> x > 10
s -> s.toUpperCase()
s -> System.out.println(s)
() -> new ArrayList<>()
```

Functional programming leads to cleaner, more maintainable, and scalable Java applications.

IMPERATIVE VS FUNCTIONAL APPROACH

The same task, expressed two different ways.

TRADITIONAL IMPERATIVE APPROACH

```
List<Integer> result = new ArrayList<>();
for (int n : numbers) {
    if (n % 2 == 0) {
        result.add(n * 2);
    }
}
```

FUNCTIONAL APPROACH WITH STREAMS

```
List<Integer> result = numbers.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

- Imperative: more code, more state, more room for errors.
- Functional: less code, easier to read, safer, and more expressive.

KEY TAKEAWAY Functional programming is not just a style — it leads to robust, scalable, future-ready software.

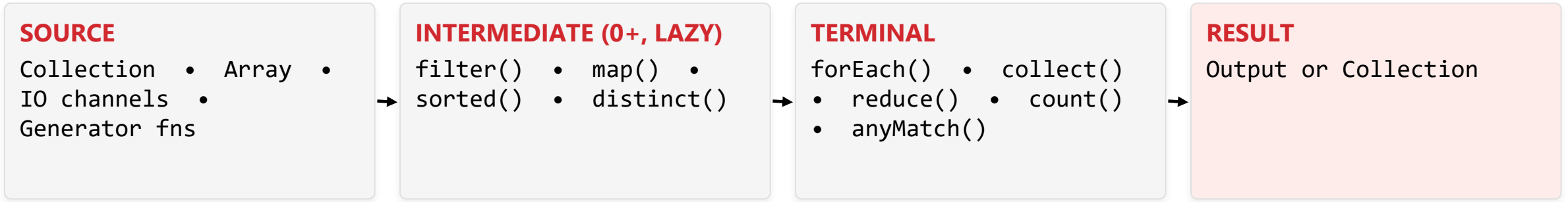
REMEMBER

Think declaratively, prefer immutability, avoid side effects, and let the stream do the work!

KEY TAKEAWAY Functional programming is a powerful approach that leads to robust, scalable, and future-ready software.

INTRODUCTION TO STREAMS API

java.util.stream provides a powerful way to process collections in a declarative, functional, and efficient manner.



Declarative

Focus on WHAT to do, not HOW.

Concise & Expressive

Write more with less code.

Performance Optimized

Internal optimizations, lazy evaluation.

Functional Style

Immutability, pure functions, no side effects.

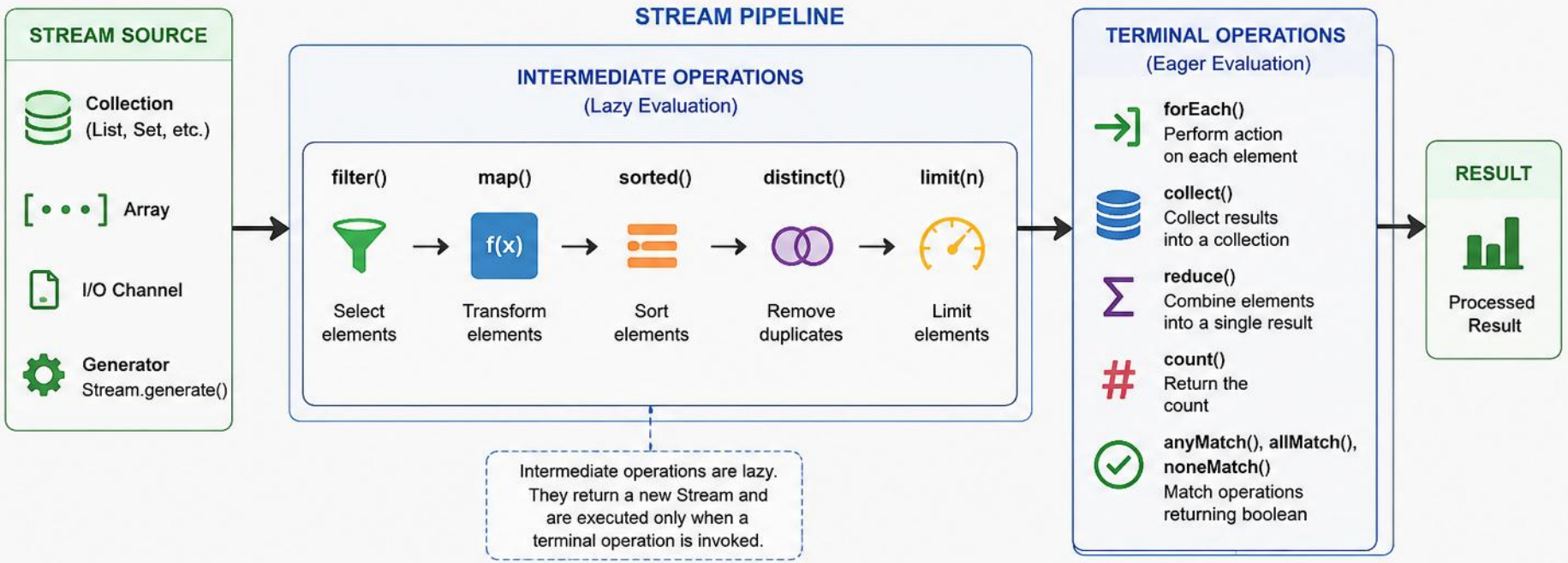
Parallel Ready

Enable with parallelStream().

KEY TAKEAWAY Streams API brings functional programming to Java, enabling powerful and efficient data processing.

JAVA STREAMS API

The Streams API (java.util.stream) provides a functional approach to process collections of data.
It enables declarative, concise and efficient operations on data sequences.



KEY CHARACTERISTICS

Lazy Evaluation
Intermediate operations are executed only when needed by a terminal operation.

Functional Style
Encourages declarative programming and avoids explicit iteration.

Pipeline Processing
Operations are chained to form a pipeline for efficient processing.

Parallel Processing
Easily switch to parallel execution using `parallelStream()`.

Immutable Source
Streams do not modify the source; they only produce results.

EXAMPLE

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David");
List<String> result = names.stream().filter(n -> n.length() > 3).map(String::toUpperCase).sorted().collect(Collectors.toList());
```

CORE CHARACTERISTICS OF STREAMS

Five properties that make streams fundamentally different from collections.

Lazy Evaluation

- Intermediate operations aren't executed until a terminal operation is invoked.

Pipeline Processing

- Operations chain to form a pipeline from source to terminal operation.

No Storage

- Streams do not store data; they process it as it flows through.

Single-Use

- A stream can be consumed only once.

Not a Data Structure

- No adding, removing, or updating elements.

KEY TAKEAWAY Streams describe a computation over data — they are not a container for the data itself.

SIMPLE EXAMPLE

Find the names of employees whose salary > 60,000, convert to upper case, and sort.

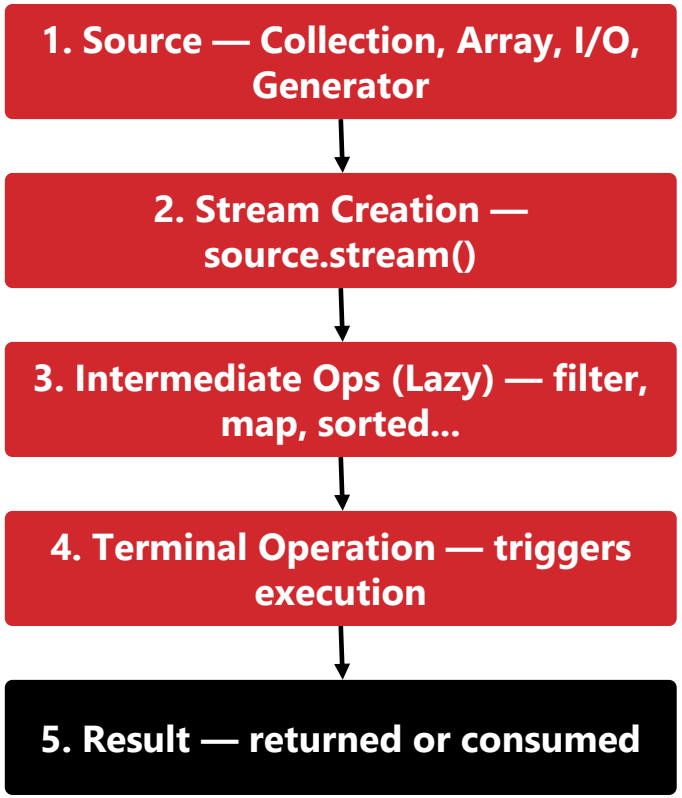
```
List<Employee> employees = ...;
List<String> result = employees.stream()
    .filter(e -> e.getSalary() > 60000)
    .map(e -> e.getName().toUpperCase())
    .sorted()
    .collect(Collectors.toList());

result.forEach(System.out::println);
```

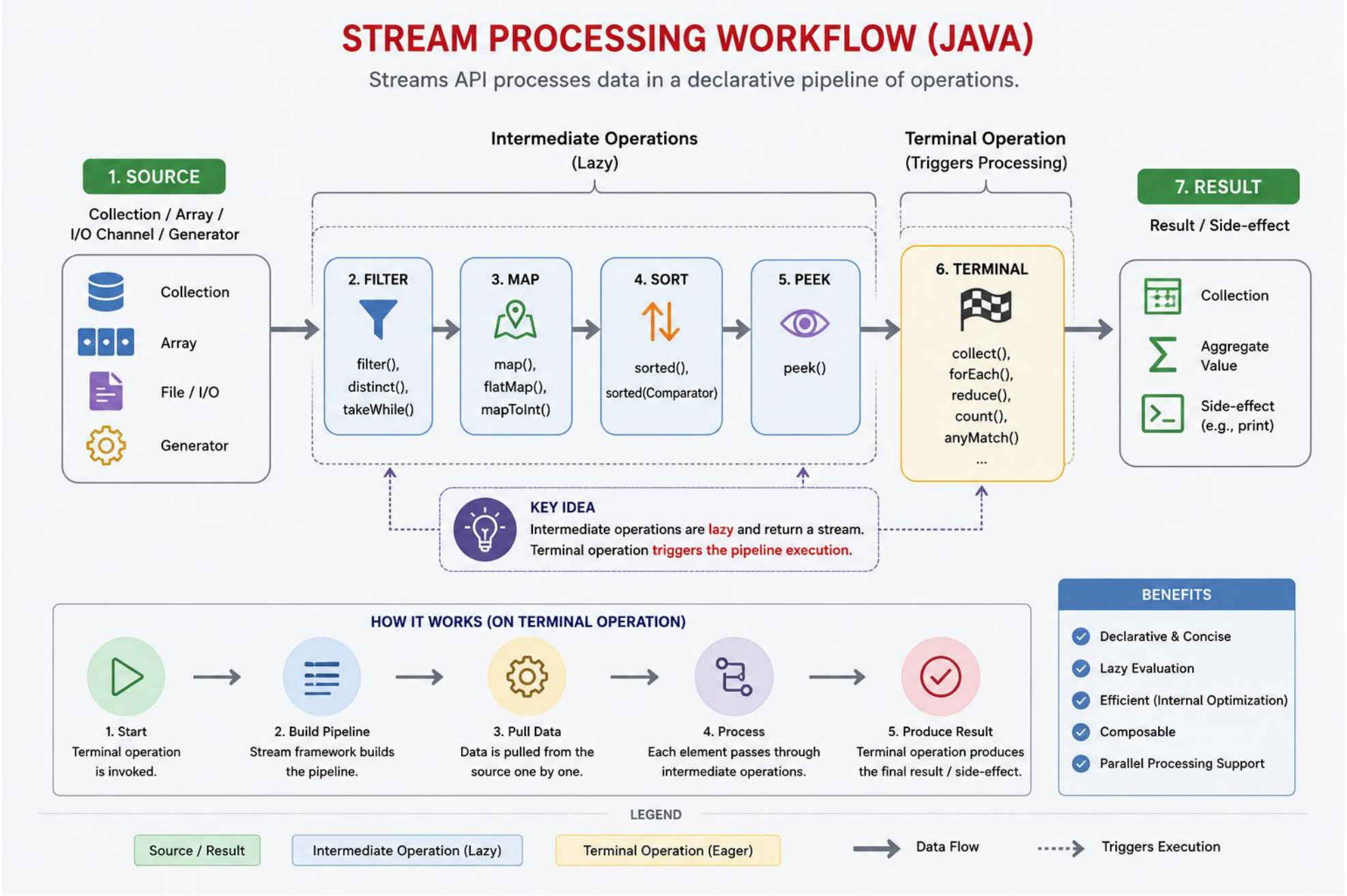
KEY TAKEAWAY Streams make complex data processing simple, readable, and maintainable.

STREAM PROCESSING WORKFLOW

Streams don't store data — they take data from a source, apply operations, and produce a result.



KEY TAKEAWAY Data flows from source to terminal operation; nothing happens until the terminal operation runs.



EXAMPLE SCENARIO

From a list of numbers: get the evens, square them, sort, and print.


```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
numbers.stream()
    .filter(n -> n % 2 == 0)       // intermediate
    .map(n -> n * n)               // intermediate
    .sorted()                      // intermediate
    .forEach(System.out::println); // terminal
```

KEY TAKEAWAY Every intermediate step in this diagram is lazy — only forEach() actually runs the pipeline.

WORKFLOW — KEY TAKEAWAYS

Five principles that apply to every stream pipeline you'll write.

- Data flows from source to terminal operation.
- Intermediate operations are lazy.
- Terminal operation triggers execution.
- Streams do not modify the source data.
- Pipeline leads to clean, efficient, and readable code.

PRO TIP: Keep intermediate operations stateless and side-effect free to fully benefit from lazy evaluation and parallel processing.

KEY TAKEAWAY Streams enable declarative, functional-style processing that is concise, expressive, and performant.

LAMBDA EXPRESSIONS

Concise syntax to represent anonymous functions — a clear, compact way to implement functional interfaces.

```
(parameter list) -> expression or statement block
// Parameters -> Arrow -> Body   (types can be inferred by the compiler)
```

- Lambda expressions implement functional interfaces.
- Improve readability and reduce boilerplate code.
- Enable functional style programming.
- Work seamlessly with Streams API, collections, and more.
- The compiler infers parameter types; types can be omitted.
- Use an expression body for single expressions; a block body for multiple statements.

KEY TAKEAWAY A lambda is just a compact, inline implementation of a functional interface's single method.

LAMBDA EXPRESSIONS IN JAVA

A lambda expression is a concise way to represent an anonymous function (a method without a name).

SYNTAX

```
(parameters) -> expression
        or
(parameters) -> { statements; }
```

STRUCTURE OF A LAMBDA EXPRESSION

PARAMETERS
Input to the lambda. Can be zero, one or more.

→

ARROW TOKEN
Separates parameters and the body.

+

BODY
Contains the expression or statements to be executed.

EXAMPLES

1. NO PARAMETERS

```
() -> "Hello World"
```

No input, returns "Hello World"

2. SINGLE PARAMETER

```
x -> x * x
```

Takes one parameter x and returns its square

3. MULTIPLE PARAMETERS

```
(a, b) -> a + b
```

Takes two parameters and returns their sum

4. MULTIPLE STATEMENTS

```
(x) -> {
    int y =x * 2;
    return y;
}
```

Body contains multiple statements

5. TYPE DECLARATION (OPTIONAL)

```
(int x) ->  x + 10
```

Type declaration is optional. Compiler can infer the type.

HOW LAMBDA EXPRESSION WORKS

FUNCTIONAL INTERFACE
An interface with a single abstract method.
e.g., `java.util.function.Function <T, R>`

Target Type →

LAMBDA EXPRESSION
`(x) -> x * x`
Provides the implementation.

→

RUNTIME
Lambda is converted to an instance of the functional interface.

→

EXECUTION
The abstract method is invoked and the lambda body is executed.

→

USES
- Collection operations
- Event handling
- Threads
- Stream API
- Callback functions

BENEFITS

- Concise and readable
- Improves productivity
- Enables functional programming
- Reduces boilerplate code

LAMBDA VS ANONYMOUS CLASS

The same behavior, five different shapes — lambdas are always shorter.

SCENARIO	LAMBDA	EQUIVALENT ANONYMOUS CLASS
No params, return a value	() -> 42	new Supplier<Integer>() {...}
One param, return a value	x -> x * x	new Function<Integer,Integer>() {...}
Multiple params, return a value	(a, b) -> a + b	new BiFunction<...>() {...}
No params, no return (void)	() -> System.out.println("Hi")	new Runnable() {...}
One param, no return (void)	msg -> System.out.println(msg)	new Consumer<String>() {...}

KEY TAKEAWAY Every lambda on the left compiles down to the same kind of object shown on the right — just with far less ceremony.

WHERE LAMBDAS ARE USED

Anywhere Java expects a functional interface.

- Collection processing (filter, map, sort, forEach)
- Event handling and callbacks
- Executors and background tasks (Runnable, Callable)
- Custom logic in APIs and frameworks
- Any place expecting a functional interface

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
names.forEach(name -> System.out.println(name.toUpperCase()));
// Prints: ALICE  BOB  CHARLIE
```

KEY TAKEAWAY A functional interface is an interface with exactly one abstract method — that's what a lambda plugs into.

FUNCTIONAL INTERFACES

An interface with exactly one abstract method (SAM) — it can also have default, static, or Object methods.

Exactly One Abstract Method

- This is the method implemented by a lambda expression.

@FunctionalInterface (Optional)

- Helps the compiler validate the rule.

Default & Static Methods

- Default methods provide reusable behavior.
- Static methods add utility related to the interface.

KEY TAKEAWAY It can also override Object methods — equals(), hashCode(), toString() — without breaking the SAM rule.

FUNCTIONAL INTERFACES IN JAVA

A Functional Interface is an interface that contains **exactly one abstract method**.
They can have any number of default, static, and Object methods.

java.lang.FunctionalInterface
(Marker Annotation)

Predicate<T>	Function<T, R>	Consumer<T>	Supplier<T>	UnaryOperator<T>	BinaryOperator<T>
boolean test(T t)	R apply(T t)	void accept(T t)	T get()	T apply(T t)	T apply(T t1, T t2)
Evaluates a condition and returns true or false.	Takes an input and produces a result.	Takes an input and performs an action.	Supplies a result without any input.	Specialization of Function with same input and output type.	Specialization of BiFunction with same input and output type.
Example: (x) -> x > 0	Example: (x) -> x.toString()	Example: (x) -> System.out.println(x)	Example: () -> new ArrayList<<)	Example: (x) -> x + 1	Example: (a, b) -> a + b

Key Characteristics

1. **Single Abstract Method** — Exactly one abstract method (SAM) defines the contract.
2. **Lambda Compatible** — Can be implemented using lambda expressions or method references.
3. **Used in Stream API** — Heavily used in Java Streams for functional-style operations.
4. **@FunctionalInterface** — Optional annotation that enables compile-time validation.

Benefits

- Promotes functional programming
- Clean and concise code
- Improves readability
- Encourages code reusability

Examples of other functional interfaces: BiFunction<T,U,R> • BiConsumer<T,U> • Runnable • Comparator<T> • Optional<T>

WHY THEY MATTER & COMMON INTERFACES

The foundation that lets you pass behavior as data — enabling lambdas, method references, and the Streams API.

INTERFACE	TYPE	ABSTRACT METHOD	DESCRIPTION
`Predicate<T>`	Function	boolean test(T t)	Evaluates a condition.
`Function<T,R>`	Function	R apply(T t)	Transforms input to output.
`BiFunction<T,U,R>`	Function	R apply(T t, U u)	Works on two inputs, produces a result.
`Consumer<T>`	Consumer	void accept(T t)	Consumes a value, returns no result.
`Supplier<T>`	Supplier	T get()	Supplies a value, takes no input.
`BiPredicate<T,U>`	Function	boolean test(T t, U u)	Evaluates a condition on two inputs.

KEY TAKEAWAY Predicate, Function, Consumer, and Supplier cover almost every functional-interface need in everyday code.

DECLARING A FUNCTIONAL INTERFACE

The @FunctionalInterface annotation is optional, but it protects you from accidentally adding a second abstract method.

```
@FunctionalInterface
public interface InterfaceName {
    returnType abstractMethod(params);
    // default, static, Object methods OK
}
```

```
@FunctionalInterface
public interface MyFunctionalInterface {
  void perform(String message);  // abstract
  default void info() {          // default
    System.out.println("Default info");
  }
}
```

KEY TAKEAWAY The abstract method is the contract; default and static methods are optional conveniences on top of it.

USING A FUNCTIONAL INTERFACE WITH A LAMBDA

The lambda supplies the implementation of the single abstract method.

```
MyFunctionalInterface obj =
    (message) -> System.out.println("Hello " + message);

obj.perform("Java");   // Output: Hello Java
```

Functional interfaces are the foundation of Java's functional programming. They allow behavior to be passed as an argument, enabling powerful and expressive code with lambdas, method references, and the Streams API.

KEY TAKEAWAY The lambda expression provides the implementation for the abstract method defined in the functional interface.

TRY IT YOURSELF — EXERCISE 1: LAMBDA EXPRESSIONS

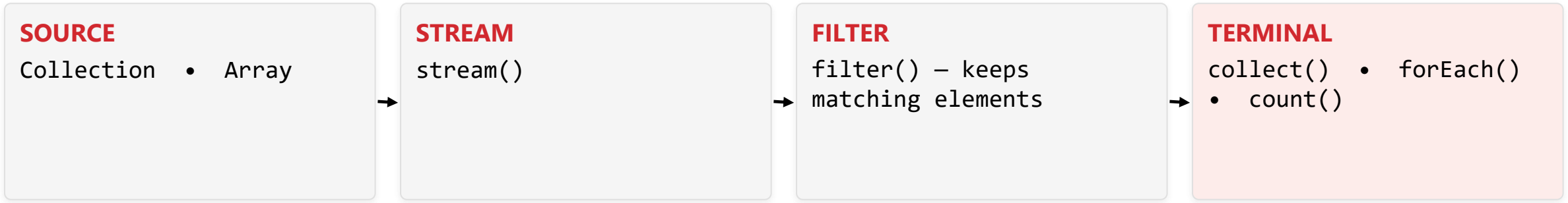
Reinforces: lambdas and functional interfaces, as just covered.

STEP	CODE	RESULT
Goal	Implement a one-method interface as an anonymous class and as a lambda	Both produce the same result
Code	SalaryCheck lambda = e -> e.salary() > 60_000;	A lambda implementing SalaryCheck.test(Employee)
Key concept	A functional interface has exactly one abstract method	That single method is what a lambda implements
Reference	exercises/exercise-01-lambda-functional-interface.md	Full starter code and steps

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-lambda-functional-interface.md.

FILTERING DATA WITH STREAMS

Filtering selects elements from a stream that match a given condition using filter().

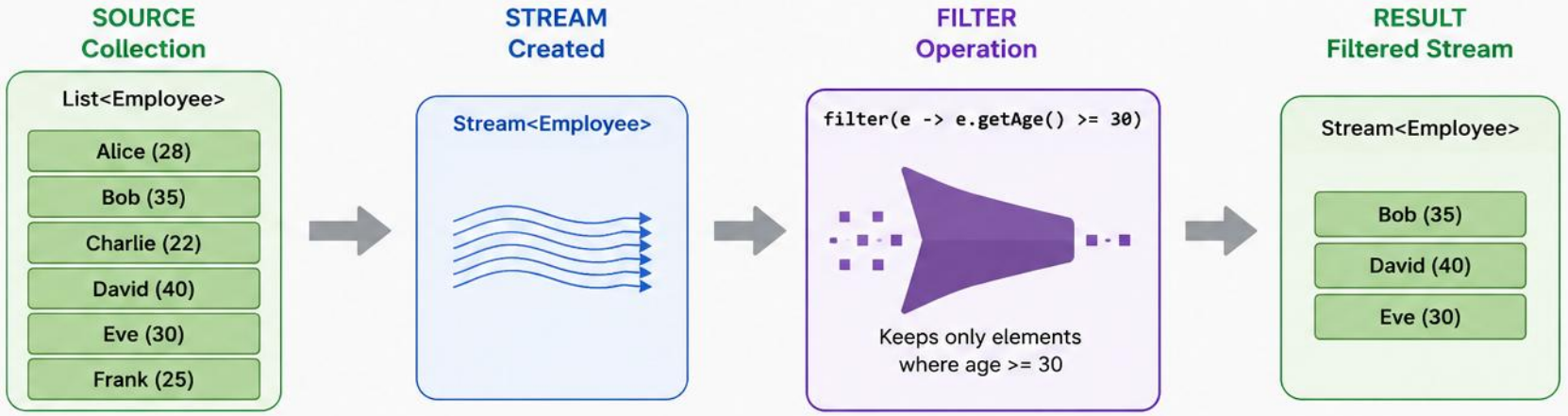


- filter() takes a Predicate and keeps only elements where it returns true.
- Other intermediate operations (map(), sorted(), distinct()) can follow it in the same pipeline.

KEY TAKEAWAY filter() is almost always the first operation you reach for in a stream pipeline.

FILTERING DATA WITH STREAMS (JAVA)

The `filter()` intermediate operation selects elements that match a given **predicate (condition)** and returns a new Stream containing only those elements.



EXAMPLE CODE

```
List<Employee> employees = Arrays.asList(
    new Employee("Alice", 28),
    new Employee("Bob", 35),
    new Employee("Charlie", 22),
    new Employee("David", 40),
    new Employee("Eve", 30),
    new Employee("Frank", 25)
);
List<Employee> filtered = employees.stream()
   .filter(e -> e.getAge() >= 30)     // Filtering condition
   .collect(Collectors.toList());     // Collect results
```

KEY POINTS

- `filter()` is an intermediate operation.
- Takes a `Predicate<T>` and returns a `Stream<T>`.
- Does not modify the original data.
- Supports method references and lambda expressions.

PREDICATE EXAMPLES

Using Lambda Expression

```
.filter(e -> e.getAge() >= 30)
```

Using Method Reference (if applicable)

```
.filter(Employee::isActive)
// where isActive() returns boolean
```

Compound Condition

```
.filter(e -> e.getAge() >= 30 && e.getSalary() > 50000)
```

COMMON FILTERING EXAMPLES

Five patterns cover most everyday filtering needs.

STREAM FILTER	PURPOSE
`filter(n -> n > 10)`	Keep numbers greater than 10
`filter(s -> s.equals("A"))`	Keep strings equal to "A"
`filter(s -> !s.isEmpty())`	Remove empty strings
`filter(n -> n % 2 == 0)`	Keep even numbers
`filter(p -> p.isActive())`	Keep active objects

```
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0)   // keep even numbers
    .collect(Collectors.toList());  // [2, 4, 6, 8, 10]
```

KEY TAKEAWAY filter() never modifies the source — it produces a new stream containing only the matching elements.

REAL-WORLD EXAMPLE: FILTER EMPLOYEES BY SALARY

Keep only employees earning more than 60,000.

NAME	DEPARTMENT	SALARY
Alice	IT	85000
Bob	HR	45000
Charlie	IT	95000
David	Finance	55000
Eve	IT	90000

```
employees.stream()
  .filter(e ->
    e.getSalary() > 60000)
  .collect(
    Collectors.toList());

// Alice, Charlie, Eve
```

KEY TAKEAWAY Use filter() to narrow your data down to only the elements that match a condition — the foundation of stream processing.

TRY IT YOURSELF — EXERCISE 2: FILTER BY SALARY

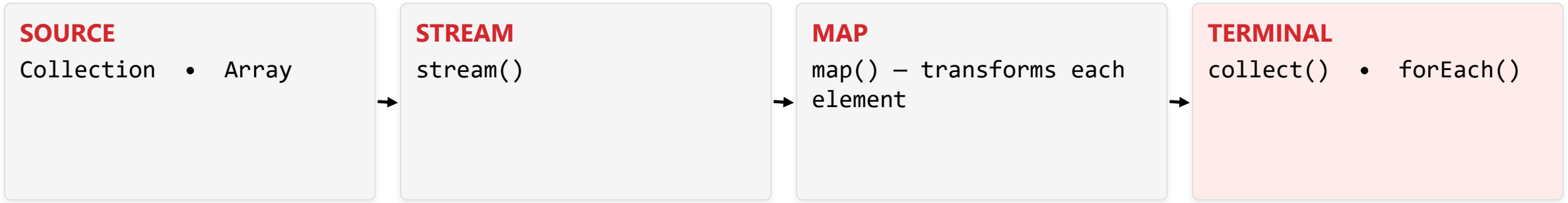
Reinforces: the filter operation you just saw.

STEP	CODE	RESULT
Goal	List employees with salary greater than 60,000	Alice, Bob, Charlie, Diana (not Evan)
Code	.filter(e -> e.salary() > 60_000)	A shorter stream — Evan is excluded
Key concept	filter() keeps only elements matching a boolean predicate	Ties to the filtering operations you just saw
Reference	exercises/exercise-02-filter-salary.md	Full starter code and steps

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-filter-salary.md.

MAPPING DATA WITH STREAMS

map() transforms each element in a stream into another form.



- Takes a function that is applied to every element in the stream.
- Produces a new stream of the resulting elements — does not modify the original data.
- Uses lazy evaluation — executed only when a terminal operation is invoked.

KEY TAKEAWAY map(x -> x * 2) turns a stream of [1,2,3,4,5] into a stream of [2,4,6,8,10] — same count, new values.

MAPPING DATA WITH STREAMS

Mapping transforms each element of a stream to another object using the `map()` or `flatMap()` operations.

SOURCE DATA

List<Employee> employees

- Employee(1, "Alice", 70000)
- Employee(2, "Bob", 60000)
- Employee(3, "Charlie", 80000)
- Employee(4, "Diana", 65000)

STREAM PIPELINE (MAPPING)

map() — Transform each Employee to Employee Name → **map()** — Transform each Name to Uppercase → **forEach()** — Consume the results

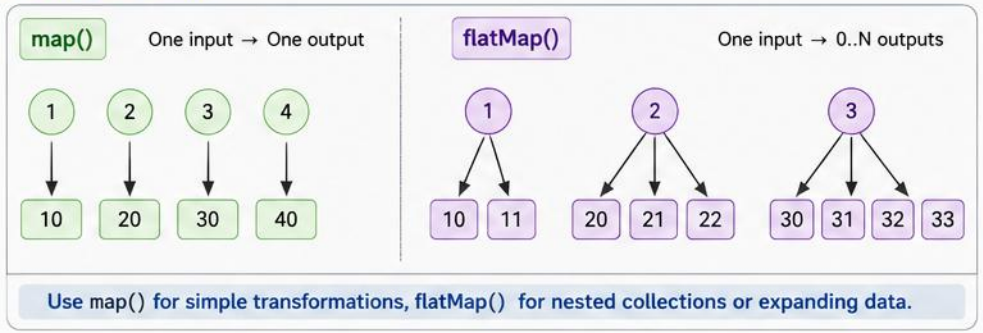
```
employees.stream()
    .map(emp -> emp.getName())      // Employee → Name
    .map(String::toUpperCase)       // Name → Uppercase
    .forEach(System.out::println); // Print result
```

MAPPED OUTPUT

Stream<String> (Uppercase Names)

- T ALICE
- T BOB
- T CHARLIE
- T DIANA

map() vs flatMap()



Use map() for simple transformations, flatMap() for nested collections or expanding data.

COMMON MAPPING EXAMPLES

Goal	Example
Extract specific field	employees.stream().map(Employee::getName)
Convert to another type	employees.stream().map(Employee::getId)
Compute derived value	employees.stream().map(emp -> emp.getSalary() * 1.1)
Flatten nested collections	departments.stream() .flatMap(dept -> dept.getEmployees().stream())

BEST PRACTICES

- Use map() for one-to-one transformation.
- Use flatMap() to flatten nested streams or collections.
- Keep mapping functions stateless and side-effect free.
- Chain multiple map() operations for complex transformations.

COMMON MAPPING EXAMPLES

One function, applied to every element.

MAPPING OPERATION	PURPOSE
`map(n -> n * 2)`	Multiply each number by 2
`map(s -> s.toUpperCase())`	Convert strings to upper case
`map(s -> s.length())`	Get length of each string
`map(d -> d.getYear())`	Extract year from each date
`map(p -> p.getName())`	Extract a property from objects

```
List<Integer> squares = numbers.stream()
    .map(n -> n * n)   // square each number
    .collect(Collectors.toList());  // [1, 4, 9, 16, 25]
```

KEY TAKEAWAY map() is a 1-to-1 transformation — every input element produces exactly one output element.

EXAMPLE: GET LENGTH OF EACH STRING

A second mapping example, plus where map() shows up in practice.

```
List<Integer> lengths = names.stream()
    .map(name -> name.length())   // length of each string
    .collect(Collectors.toList());  // [5, 3, 7]
```

- Common use cases: data transformation, format conversion, extracting properties,
- calculations, and preparing data for other operations in the pipeline.

KEY TAKEAWAY Use map() to transform each element into a new form — a powerful tool for shaping data for the next pipeline step.

TRY IT YOURSELF — EXERCISE 3: LIST ALL NAMES

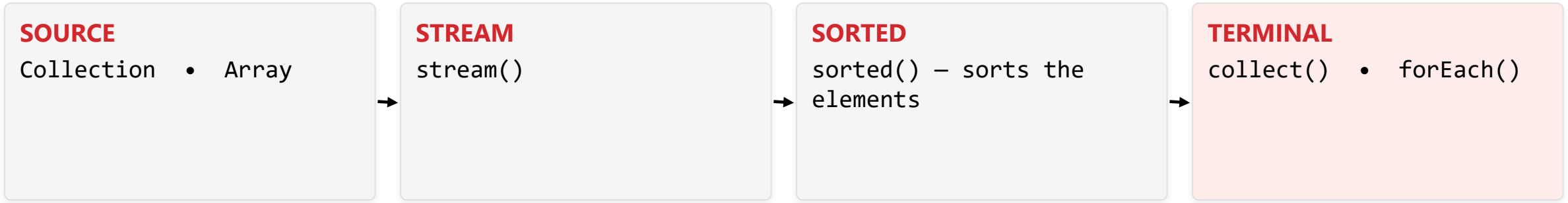
Reinforces: the map operation you just saw.

STEP	CODE	RESULT
Goal	Print all employee names using map + forEach or collect	All five names print
Code	.map(Employee::name).forEach(System.out::println)	Alice, Bob, Charlie, Diana, Evan
Key concept	map() transforms each element — Employee to String here	Ties to the mapping operations you just saw
Reference	exercises/exercise-03-list-names.md	Full starter code and steps

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-list-names.md.

SORTING WITH STREAMS

sorted() arranges the elements of a stream in a specific order.



- Returns a new stream with elements in sorted order — does not modify the source.
- By default, sorts in natural (ascending) order for Comparable elements.
- You can provide a custom Comparator to define your own order.

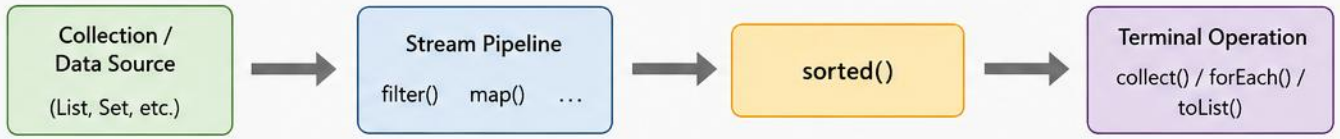
KEY TAKEAWAY sorted() on [7,3,1,9,5] produces [1,3,5,7,9] — natural ascending order by default.

SORTING WITH STREAMS IN JAVA

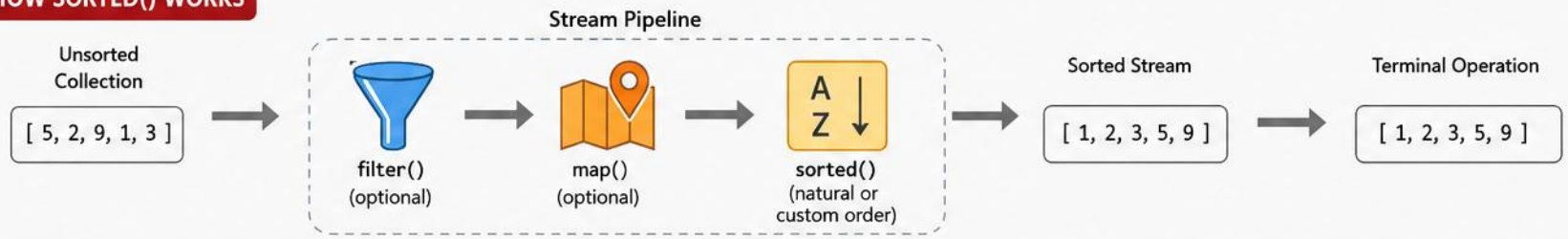
The Stream API provides the **sorted()** operation to sort elements in natural order or using a custom Comparator.

1. OVERVIEW

sorted() is an intermediate operation that returns a stream with elements sorted in natural order or by a provided Comparator.



2. HOW SORTED() WORKS



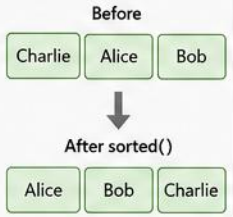
3. SORTING EXAMPLES

A. Natural Sorting (Comparable)

```
List<String> names = List.of("Charlie", "Alice", "Bob");

List<String> sorted = names.stream()
            .sorted()
            .toList();


// Output: [Alice, Bob, Charlie]
```

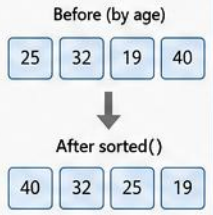


B. Custom Sorting (Comparator)

```
List<Person> people = ...;

List<Person> sorted = people.stream()
    .sorted(Comparator.comparing(Person::getAge)
            .reversed())   // Descending by age
    .toList();


// Output: Sorted by age (highest to lowest)
```



COMPARATOR EXAMPLES

- **Natural Order** — sorted()
- **Reverse Order** — sorted(Comparator.reverseOrder())
- **By Property** — sorted(Comparator.comparing(Person::getAge))
- **Multiple Fields** — sorted(Comparator.comparing(Person::getAge).thenComparing(Person::getName))

4. KEY POINTS

- sorted() is stable (maintains order of equal elements).
- Natural sorting requires elements to implement Comparable.
- Use Comparator for custom sorting logic.
- Can be combined with other operations in the pipeline.
- Returns a new sorted stream; original data remains unchanged.

COMMON SORTING EXAMPLES

Natural order, reverse order, and custom comparators.

EXAMPLE	DESCRIPTION	RESULT
`sorted()`	Natural order (Comparable types)	[1,2,3,4,5]
`sorted(Comparator.reverseOrder())`	Reverse order	[5,4,3,2,1]
`sorted(String.CASE_INSENSITIVE_ORDER)`	Case-insensitive string sort	[alice,Bob,charlie]
`sorted(Comparator.comparing(p -> p.name()))`	Custom sort by property (asc)	[Alice,Bob,Charlie]

```
List<Integer> sorted = numbers.stream()
    .sorted()                          // natural ascending order
    .collect(Collectors.toList());     // [1, 3, 5, 7, 9]
```

KEY TAKEAWAY For custom classes, implement Comparable<T> or provide a Comparator to sorted().

EXAMPLE: SORT OBJECTS BY SALARY (DESCENDING)

Comparator.comparing(...).reversed() sorts custom objects in descending order.

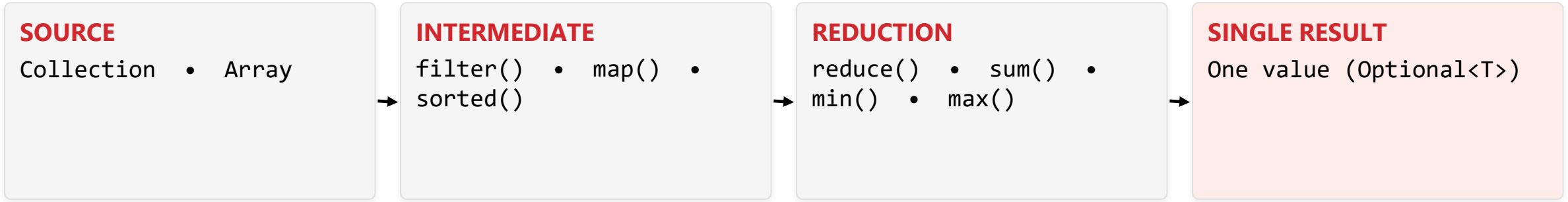
```
List<Employee> sortedBySalary = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary)
        .reversed())        // descending by salary
    .collect(Collectors.toList());
```

NAME	SALARY (DESC)
Bob	95000
Alice	75000
Charlie	55000

KEY TAKEAWAY Use sorted() with natural ordering or a custom Comparator for flexible, expressive sorting.

REDUCTION OPERATIONS

Reduction combines the elements of a stream into a single result using an accumulator function.



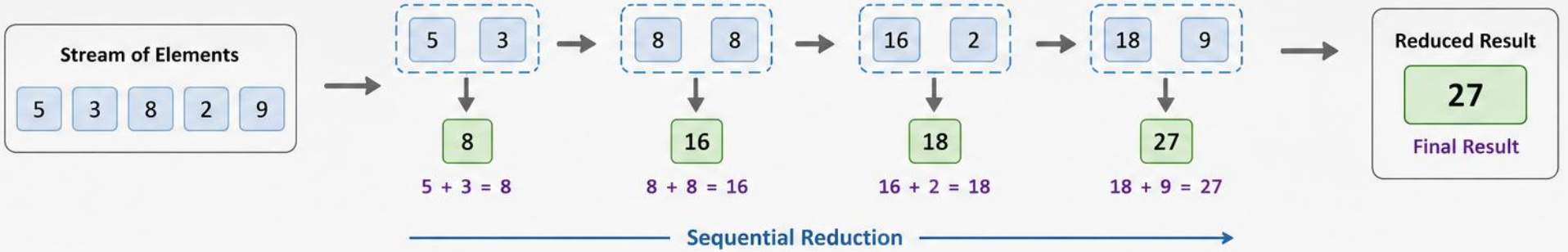
- Uses an identity value (initial value) and an accumulator function, applied between the current result and each element.
- The accumulator must be associative and stateless. Optional<T> is returned when a reduction may have no result.

KEY TAKEAWAY Common reductions: sum, min, max, count, average, product, and custom logic.

REDUCTION OPERATIONS IN JAVA

Reduction operations combine the elements of a stream into a single result using an identity value and an associative accumulation function.

HOW REDUCTION WORKS



TYPES OF REDUCTION (Stream API)

1 reduce(identity, accumulator)

Combines elements with an identity value.
Returns identity if stream is empty.

```
Optional<T> reduce(T identity,
          BinaryOperator<T> accumulator)
```

2 reduce(accumulator)

Combines elements without an identity.
Returns `Optional.empty()` if stream is empty.

```
Optional<T> reduce(BinaryOperator<T> accumulator)
```

3 reduce(identity, accumulator, combiner)

Used in parallel streams. Combines partial results using combiner.

```
T reduce(T identity, BinaryOperator<T> accumulator,
          BinaryOperator<T> combiner)
```

COMMON EXAMPLES

		Result
Σ	**Sum** `reduce(0, Integer::sum)`	27
×	**Product** `reduce(1, (a, b) -> a * b)`	2160
max	**Maximum** `reduce(Integer::max)`	9
min	**Minimum** `reduce(Integer::min)`	2
concat	**Concatenate Strings** `reduce("", (a, b) -> a + b)`	"53829"

CODE EXAMPLE

```
List<Integer> numbers = Arrays.asList(5, 3, 8, 2, 9);

// 1. With identity
int sum = numbers.stream()
       .reduce(0, Integer::sum);
System.out.println("Sum = " + sum);  // 27

// 2. Without identity
Optional<Integer> max = numbers.stream()
       .reduce(Integer::max);
max.ifPresent(m -> System.out.println("Max = " + m)); // 9

// 3. Parallel stream with combiner
int sumParallel = numbers.parallelStream()
       .reduce(0, Integer::sum, Integer::sum);
System.out.println("Parallel Sum = " + sumParallel); // 27
```

KEY POINTS

- Reduction produces a single summary result from the stream.
- The accumulator function must be associative.
- Use identity when a default starting value exists.
- For parallel streams, provide a proper combiner function.

Works with Sequential & Parallel Streams

REDUCTION METHODS OVERVIEW

Eight methods, each with a distinct return type.

METHOD	RETURN TYPE	DESCRIPTION	EXAMPLE
`reduce(identity, accumulator)`	T	Combines using identity + accumulator	reduce(0, Integer::sum)
`reduce(accumulator)`	Optional<T>	Combines without an identity value	reduce(Integer::sum)
`sum()`	int/long/double	Sum of numeric stream elements	mapToInt(i->i).sum()
`min(comparator) / max(comparator)`	Optional<T>	Smallest / largest element	min(Comparator.naturalOrder())
`count()`	long	Number of elements	count()
`average()`	OptionalDouble	Average of numeric elements	mapToInt(i->i).average()
`collect(collector)`	R	Reduces using a Collector	collect(Collectors.toList())

KEY TAKEAWAY Know your return types: Optional<T>, OptionalDouble, long, int — each reduction returns a different shape.

HOW REDUCTION WORKS

reduce(0, Integer::sum) on the stream 2, 4, 6, 8, 10.

Final Result = 30

KEY TAKEAWAY At each step, the accumulator function is applied between the current result and the next element.

REDUCTION — CODE EXAMPLES

reduce, min, max, count, and average, all on the same data.

```
List<Integer> numbers = Arrays.asList(2, 4, 6, 8, 10);

int sum = numbers.stream().reduce(0, Integer::sum);          // 30
Optional<Integer> sumOpt = numbers.stream().reduce(Integer::sum);  // Optional[30]
Optional<Integer> min = numbers.stream().min(Integer::compare);    // Optional[2]
Optional<Integer> max = numbers.stream().max(Integer::compare);    // Optional[10]
long count = numbers.stream().count();                             // 5
OptionalDouble avg = numbers.stream()
    .mapToInt(Integer::intValue).average();  // OptionalDouble[6.0]
```

KEY TAKEAWAY reduce(identity, accumulator) always returns T; reduce(accumulator) returns Optional<T> to handle empty streams.

COMMON REDUCTIONS & CUSTOM REDUCTION

Built-in reductions, plus a hand-written one for string concatenation.

OPERATION	EXAMPLE	DESCRIPTION	RESULT
Sum	mapToInt(i->i).sum()	Sum of all elements	30
Min / Max	min/max(Integer::compare)	Smallest / largest element	2 / 10
Product	reduce(1, (a,b)->a*b)	Product of elements	3840

```
// Goal: concatenate all strings with commas
String result = names.stream()
    .reduce("", (a, b) -> a.isEmpty() ? b : a + ", " + b);

System.out.println(result);   // Alice, Bob, Charlie
```

KEY TAKEAWAY For custom reductions: accumulator must be associative, avoid modifying external state, use a neutral identity.

REDUCTION — BEST PRACTICES

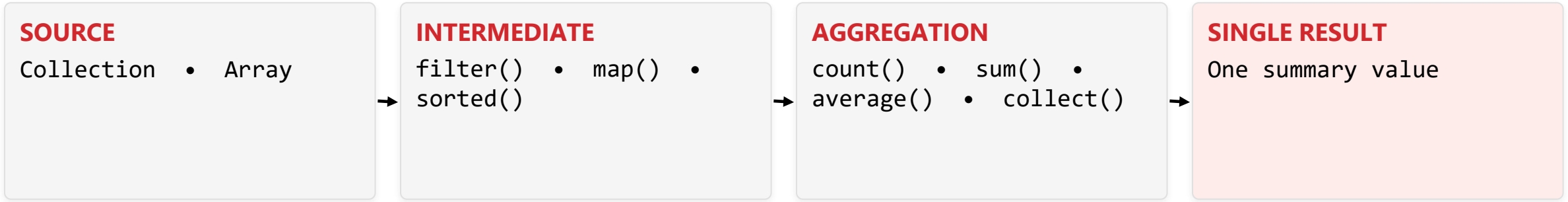
Five guidelines for choosing and writing reductions correctly.

- Use built-in reduction methods (sum, min, max, count, average) whenever possible.
- Use reduce(identity, accumulator) when a natural identity exists (0 for sum, 1 for product, "" for concatenation).
- Use reduce(accumulator) when no identity value exists or when working with objects.
- Ensure the accumulator is associative and stateless for correct parallel execution.
- Prefer Optional<T> results to safely handle empty streams.

KEY TAKEAWAY Think in terms of identity, accumulator, and associativity — reduction operations are essential for aggregation and summarization.

AGGREGATION OPERATIONS

Aggregation computes summary information from a stream — count, sum, average, min, max, and more.

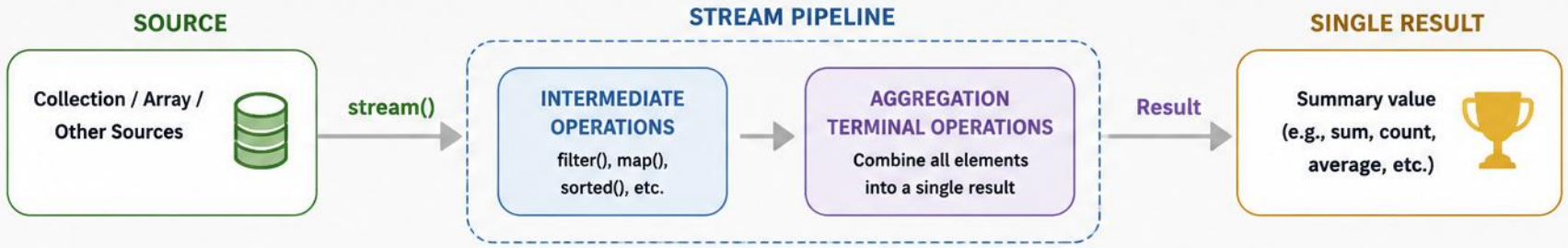


- Aggregation summarizes data without changing the source stream — it only computes results.
- Aggregations are terminal operations, used for reporting, analytics, dashboards, and business insights.

KEY TAKEAWAY For numeric types, use the correct return type: int, long, double, or Optional<T>.

AGGREGATION OPERATIONS IN JAVA STREAMS

Combine multiple elements into a single summary result



COMMON AGGREGATION OPERATIONS

SUM

Returns the sum of numeric elements.

```
int sum = numbers.stream()
    .mapToInt(Integer::intValue)
    .sum();
```

COUNT

Returns the count of elements.

```
long count = items.stream()
    .count();
```

AVERAGE

Returns the average of numeric elements.

```
double avg = numbers.stream()
    .mapToDouble(Integer::doubleValue)
    .average()
    .orElse(0.0);
```

MIN

Returns the minimum element.

```
Optional<Integer> min = numbers.stream()
    .min(Integer::compare);
```

MAX

Returns the maximum element.

```
Optional<Integer> max = numbers.stream()
    .max(Integer::compare);
```

REDUCE

Combines elements using an accumulator function.

```
int total = numbers.stream()
    .reduce(0, (a, b) -> a + b);
```

COLLECT

Aggregates elements into a collection or summary object.

```
List<String> list = names.stream()
    .collect(Collectors.toList());
```

GROUPING & SUMMARIZING (Collectors)

Performs advanced aggregations like grouping, partitioning, and summarizing.

```
Map<String, Long> byCity = people.stream()
    .collect(Collectors.groupingBy(
     Person::getCity, Collectors.counting()));
```

Key Points

- Aggregation operations produce a single result.
- They are terminal operations (pipeline ends here).
- Used for calculations, summaries, and data analysis.
- Provide better readability and concise code.

AGGREGATION OPERATIONS OVERVIEW

Seven operations, from a simple count() to full summaryStatistics().

OPERATION	RETURN TYPE	DESCRIPTION	EXAMPLE
`count()`	long	Counts the number of elements	stream.count()
`sum()`	int/long/double	Sum of numeric elements	mapToInt(...).sum()
`average()`	OptionalDouble	Average of elements	mapToDouble(...).average()
`min() / max()`	Optional<T>	Minimum / maximum element	min(naturalOrder())
`summaryStatistics()`	*SummaryStatistics	Count/sum/min/max/average together	mapToInt(...).summaryStatistics()
`collect()`	R	Custom aggregation using collectors	collect(summingInt(...))

KEY TAKEAWAY summaryStatistics() is the one to reach for when you need several metrics at once, in a single pass.

HOW AGGREGATION WORKS

Given the stream 2, 4, 6, 8, 10 — every metric computed in one pass.

METRIC	VALUE
Count	5
Sum	30
Min	2
Average	6.0
Max	10

```
IntSummaryStatistics stats =
    numbers.stream()
    .mapToInt(i -> i)
    .summaryStatistics();

stats.getCount();    // 5
stats.getSum();      // 30
stats.getMin();      // 2
stats.getAverage();  // 6.0
stats.getMax();      // 10
```

KEY TAKEAWAY Aggregation condenses many values into meaningful insights that help in decision making.

CUSTOM AGGREGATION EXAMPLE

Calculate the total salary of all employees using a Collector.

```
double totalSalary = employees.stream()
    .collect(Collectors.summingDouble(Employee::getSalary));
System.out.println(totalSalary);   // Output: 225000.0
```

- Collectors.counting() — counts elements. Collectors.summingInt/Long/Double() — sums values.
- Collectors.averagingInt/Double() — averages values. Collectors.minBy()/maxBy() — min/max using a Comparator.
- Collectors.summarizingInt/Double() — all summary statistics together.

KEY TAKEAWAY Prefer Collectors for custom or complex aggregations rather than hand-rolling a reduce().

AGGREGATION — BEST PRACTICES

Five guidelines for accurate, efficient aggregation.

- Choose the right aggregation method for the data type.
- Use mapToInt(), mapToLong(), or mapToDouble() before sum()/average() for numeric streams.
- Handle Optional results from min(), max(), and average() safely.
- Use summaryStatistics() when multiple metrics are needed together.
- Aggregation operations are terminal — they produce a single result.

KEY TAKEAWAY Aggregation helps in analytics, reporting, and decision making — pick the right operation and return type.

TRY IT YOURSELF — EXERCISE 4: HIGHEST AND LOWEST SALARY

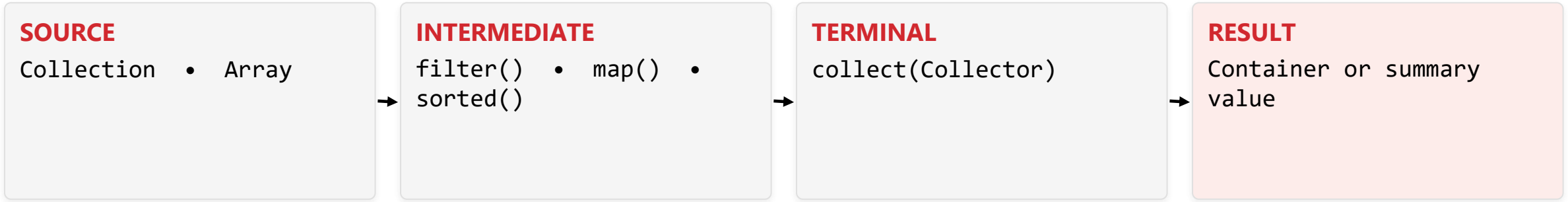
Reinforces: the aggregation operations you just saw.

STEP	CODE	RESULT
Goal	Find max and min salary with Comparator.comparingDouble	Max Diana 90000; min Evan 55000
Code	.max(comparingDouble(Employee::salary)).orElseThrow()	max()/min() return an Optional<Employee>
Key concept	orElseThrow() unwraps the Optional or fails fast if empty	Ties to the aggregation operations you just saw
Reference	exercises/exercise-04-minmax.md	Full starter code and steps

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-minmax.md.

COLLECTORS FRAMEWORK

A rich set of collector implementations to accumulate stream elements into result containers or summary values.



- Collectors are immutable, stateless, and thread-safe — all defined in java.util.stream.Collectors.
- Common use cases: grouping, partitioning, summarizing, joining, and collecting into collections.

KEY TAKEAWAY collect(Collector) is the terminal operation that triggers the pipeline and returns the result.

HOW COLLECTORS WORK

Collectors.toSet() on ["Alice","Bob","Charlie","Bob"] — duplicates are removed.


```
List<Integer> list = numbers.stream().collect(Collectors.toList());   // [1,2,3,4,5]
Set<Integer> set = numbers.stream().collect(Collectors.toSet());      // {1,2,3,4,5}
String joined = numbers.stream().map(Object::toString)
    .collect(Collectors.joining(", "));   // "1, 2, 3, 4, 5"
```

KEY TAKEAWAY A collector defines how to accumulate elements, the result type, and how partial results combine in parallel.

COMMON COLLECTORS — PART 1

Collection and reduction collectors.

CATEGORY	COLLECTOR	DESCRIPTION	EXAMPLE
`Collection`	toList() / toSet()	Collects into a List / Set	collect(toList())
`Collection`	toCollection(Supplier)	Collects into a specific collection	toCollection(ArrayList::new)
`Reduction`	counting()	Counts the number of elements	collect(counting())
`Reduction`	summingInt() / averagingDouble()	Sums / averages values	summingInt(Employee::getSalary)
`Reduction`	minBy() / maxBy()	Min/max element using a Comparator	minBy(naturalOrder())

KEY TAKEAWAY count=5, sum=15, min=1, average=3.0, max=5 — that's what Collectors.summarizingInt() gives you in one call.

COMMON COLLECTORS — PART 2 & GROUPING EXAMPLE

Grouping and joining collectors, plus groupingBy() in action.

CATEGORY	COLLECTOR	DESCRIPTION	EXAMPLE
Grouping	groupingBy(...)	Groups elements by a classifier function	groupingBy(Employee::getDepartment)
Grouping	partitioningBy(...)	Partitions into two groups (true/false)	partitioningBy(e -> e.getAge()>30)
Joining	joining(delim, prefix, suffix)	Joins with delimiter, prefix, and suffix	joining(", ", "[", "]")

```
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Result: HR -> [Alice, Charlie]   IT -> [Bob, David]
```

KEY TAKEAWAY groupingBy() is the collector you'll reach for most in real business reporting.

COLLECTORS — BEST PRACTICES

Six guidelines for choosing the right collector.

- Use toList(), toSet(), or toMap() for direct collection needs.
- Use groupingBy() and partitioningBy() for data categorization.
- Use summarizing() collectors for statistical insights.
- Use toUnmodifiableList/Set/Map() (Java 10+) for immutable results.
- Collectors are safe for parallel streams.

```
Popular import: import static java.util.stream.Collectors.*;
```

KEY TAKEAWAY The Collectors framework transforms stream data into collections, summaries, and structured results efficiently.

TRY IT YOURSELF — EXERCISE 5: MAP 10% RAISE

Reinforces: map() and collecting a stream into a List, as just covered.

STEP	CODE	RESULT
Goal	Map salaries ×1.10, collect into a List<Double>	Updated salary list prints
Code	.map(e -> e.salary() * 1.10).toList()	A new List<Double> of raised salaries
Key concept	toList() is a terminal op that collects the stream into a List	Ties to the Collectors framework you just saw
Reference	exercises/exercise-05-map-raise.md	Full starter code and steps

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-map-raise.md.

TRY IT YOURSELF — EXERCISE 6: COUNT BY DEPARTMENT

Reinforces: groupingBy and counting, as just covered.

STEP	CODE	RESULT
Goal	Count employees per department with groupingBy + counting	Finance=1, HR=2, IT=2 (TreeMap: alphabetical)
Code	groupingBy(Employee::department, counting())	A Map<String, Long> of counts
Key concept	groupingBy buckets elements by a classifier function	Ties to the Collectors framework you just saw
Reference	exercises/exercise-06-group-count.md	Full starter code and steps

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-group-count.md.

STREAM PIPELINES

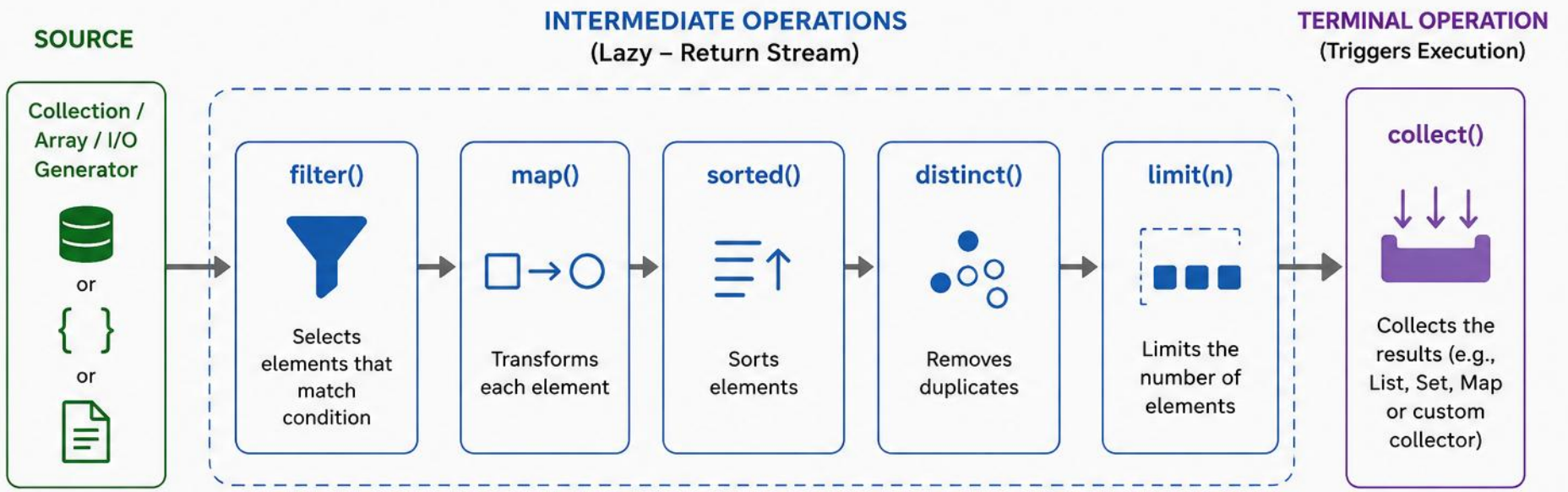
A sequence of operations on a stream: a source, zero or more intermediate operations, and a terminal operation.

STAGE	PURPOSE
`Source`	Get a stream from the data source.
`Intermediate Operation`	Transform or filter the stream (lazy).
`Intermediate Operation`	Apply additional transformations or filtering (lazy).
`Terminal Operation`	Produce a result or side effect (eager).
`Result`	Collection, value, or side effect.

KEY TAKEAWAY Pipelines are designed for readability, composability, and parallelism — and never modify the source data.

STREAM PIPELINES IN JAVA

A Stream Pipeline consists of a data source, zero or more intermediate operations, and a terminal operation. Intermediate operations are lazy and return a stream. The terminal operation triggers the pipeline execution.



Lazy Evaluation
Intermediate operations do not process data until needed.

Pipelining
Operations are linked together to form a pipeline.

Single Pass
Data flows through the pipeline once, improving performance.

Optimization
Streams enable internal optimizations like short-circuiting.

Example Pipeline

```
List<String> names = ...

names.stream()
    .filter(s -> s.startsWith("A"))
    .map(String::toUpperCase)
    .distinct()
    .sorted()
    .collect(Collectors.toList());
```

HOW IT WORKS: HR NAMES, SORTED

Get names of HR employees, sort them, and collect into a list.

STAGE	ACTION
`Source`	employees (List<Employee>)
`filter()`	Keep only employees from the HR department
`map()`	Extract employee names
`sorted()`	Sort names in natural order
`collect()`	Collect into a List<String> (Result)

```
List<String> hrNames = employees.stream()
    .filter(e -> e.getDept().equals("HR")).map(Employee::getName)
    .sorted().collect(Collectors.toList());  // [Alice, Charlie]
```

KEY TAKEAWAY The terminal operation collect() is what triggers the entire pipeline to actually run.

INTERMEDIATE VS TERMINAL OPERATIONS

Two very different roles in the same pipeline.

Intermediate Operations

- Return another stream.
- Can be chained.
- Lazy — not executed until a terminal operation is called.
- Examples: filter(), map(), sorted(), distinct(), limit()

Terminal Operations

- Produce a result or side effect.
- End the pipeline.
- Trigger processing of all operations in the pipeline.
- Examples: forEach(), collect(), count(), reduce(), findFirst()

KEY TAKEAWAY Intermediate operations are lazy; terminal operations are eager — this is the single most important distinction in this module.

TYPES OF STREAM OPERATIONS

Stateless, stateful, and short-circuiting — three categories with very different costs.

TYPE	DESCRIPTION	CHARACTERISTICS	EXAMPLES
`Stateless`	Depend only on the current element.	Easier to parallelize, no state retained.	map(), filter(), sorted()*, distinct()*
`Stateful`	Depend on multiple elements or require state.	May require buffering, costlier.	sorted(), distinct(), limit(), skip()
`Short-Circuiting`	May terminate early without processing all elements.	Improves performance, reduces computation.	findFirst(), anyMatch(), allMatch(), limit()

KEY TAKEAWAY distinct() is stateful because it must remember every previously seen element to detect duplicates.

PIPELINES — BEST PRACTICES

Four habits for readable, efficient pipelines.

- Build pipelines that are readable and self-explanatory.
- Prefer inexpensive intermediate operations before expensive ones (filter() before sorted()).
- Avoid side effects in intermediate operations; keep them stateless and non-interfering.
- Use parallel streams for large datasets when appropriate.

```
Popular Entry Point:  Stream<T> stream()  – the entry point for starting a stream pipeline.
```

KEY TAKEAWAY Stream pipelines improve code readability, reduce boilerplate, and enable parallel execution.

TRY IT YOURSELF — EXERCISE 7: HR DEPARTMENT NAMES

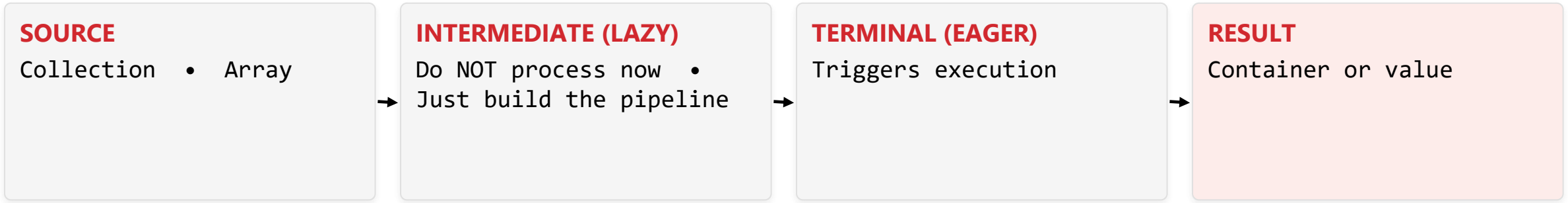
Reinforces: chaining filter and map into one pipeline.

STEP	CODE	RESULT
`Goal`	Get the names of employees in the HR department	Alice, Charlie
`Code`	filter(department == HR) → map(name)	filter narrows first, then map transforms
`Key concept`	Multiple intermediate ops can chain before the terminal one	Ties to the pipeline example you just saw
`Reference`	exercises/exercise-07-hr-names.md	Full starter code and steps

TRY IT NOW Complete Exercise 7 before continuing — see exercises/exercise-07-hr-names.md.

LAZY EVALUATION

Stream operations aren't executed until the terminal operation is invoked — intermediate operations are only recorded.

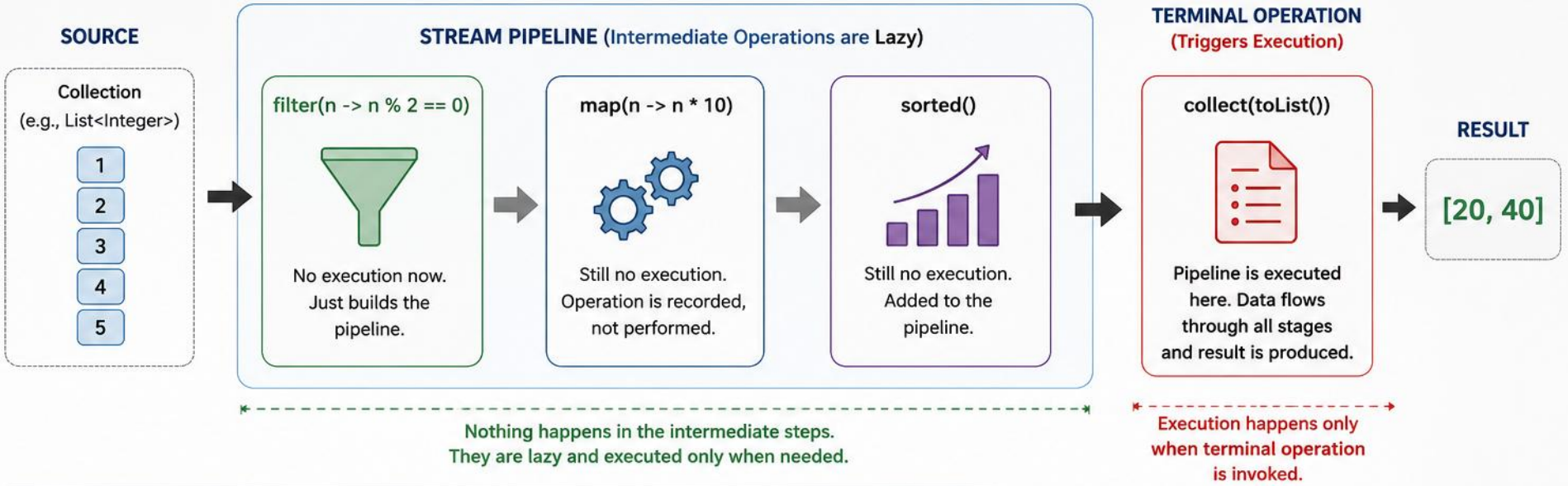


- The pipeline is executed only when a terminal operation is called; data flows element by element, on demand.
- Lazy evaluation avoids unnecessary computation, improves performance, and enables short-circuiting.

KEY TAKEAWAY Nothing happens until a terminal operation is called — that's the whole idea of lazy evaluation.

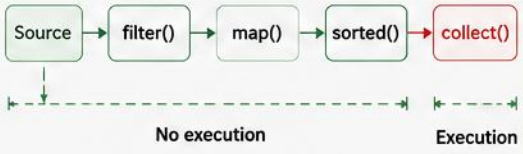
LAZY EVALUATION IN JAVA

Intermediate operations are not executed until a terminal operation is invoked.



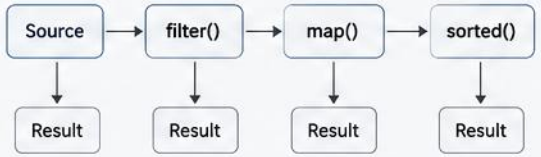
LAZY EVALUATION vs EAGER EVALUATION

LAZY EVALUATION (Streams)



- Intermediate operations are lazy.
- Execution happens only at terminal operation.
- Better performance and reduced memory usage.
- Enables processing large or infinite data.

EAGER EVALUATION (Traditional Loops)



- Each operation is executed immediately.
- Intermediate results are stored in memory.
- Higher memory usage and lower performance.
- Not suitable for large or infinite data.

KEY TAKEAWAYS

- Streams use lazy evaluation.
- It builds a pipeline of operations.
- No data is processed until a terminal operation is called.
- This leads to better performance and efficient resource utilization.

```
// Example
List<Integer> result = numbers.stream()
      .filter(n  -> n % 2 == 0)
      .map(n  -> n * 10)
      .sorted()
      .collect(Collectors.toList()); // Triggers execution
```

BEFORE AND AFTER COLLECT()

Building the pipeline is free; running it costs exactly one pass through the data.

Before collect()

- numbers.stream().filter(...).map(...).sorted(...)
- No data is processed.
- Intermediate operations are recorded.
- Result is another Stream.

When collect() Executes

- Terminal operation is invoked.
- The entire pipeline runs.
- Data flows element by element through all stages.
- Final result is produced and returned.

KEY TAKEAWAY A pipeline built but never given a terminal operation simply never runs — Java won't warn you either.

PIPELINE ANALOGY

Imagine a water pipeline: pipes are connected, but water doesn't flow until the tap opens.

- Pipes are connected when the pipeline is built — but nothing flows yet.
- Water (data) only flows once collect() opens the tap.

KEY TAKEAWAY Pipes are connected when the pipeline is built; water only flows when collect() opens the tap.

CODE EXAMPLE & OUTPUT

Print statements inside filter()/map() reveal exactly when each element is processed.

```
numbers.stream()
  .filter(n -> {
    System.out.println("filter: " + n);
    return n % 2 == 0;
  })
  .map(n -> {
    System.out.println("map: " + n);
    return n * 10;
  })
  .sorted();   // no terminal op yet

result.collect(Collectors.toList());
```

```
--- Building pipeline ---
--- Pipeline built, no output above ---
--- Now invoking terminal operation ---
filter: 1
filter: 2
map: 2
filter: 3
filter: 4
map: 4
filter: 5
Result: [20, 40]
```

KEY TAKEAWAY Not a single print statement runs until collect() is called — proof that intermediate operations are lazy.

LAZY VS EAGER BEHAVIOR

Streams vs traditional collections/loops, side by side.

ASPECT	LAZY EVALUATION (STREAMS)	EAGER EVALUATION (LOOPS)
`Execution Time`	At terminal operation	Immediately
`Intermediate Ops`	Not executed until needed	Executed right away
`Memory Usage`	More efficient (on demand)	May use more memory
`Short-Circuiting`	Possible (findFirst, anyMatch)	Not possible
`Performance`	Better for large datasets	May do unnecessary work

KEY TAKEAWAY Lazy evaluation is precisely what makes short-circuiting operations like findFirst() possible.

SHORT-CIRCUITING BENEFITS

Operations that can stop before processing the whole stream.

OPERATION	DESCRIPTION	EXAMPLE BEHAVIOR
`findFirst()` / `findAny()`	Returns the first / any matching element	Stops when a match is found
`anyMatch()`	Tests whether any element matches	Stops when a match is found
`allMatch()` / `noneMatch()`	Tests whether all / no elements match	Stops on first failure / match
`limit(n)`	Limits the number of elements	Stops after n elements
`count()`	Counts all elements	Processes the entire stream (not short-circuiting)

KEY TAKEAWAY Use short-circuiting operations when possible — they can turn an O(n) scan into an early exit.

LAZY EVALUATION — BEST PRACTICES

Build the pipeline now, execute it later.

- Remember that intermediate operations are lazy.
- Use terminal operations to trigger execution.
- Use short-circuiting operations when possible for better performance.
- Avoid side effects in intermediate operations.
- Streams are ideal for large or infinite data sources.

KEY TAKEAWAY Lazy evaluation defers computation until the last possible moment — the terminal operation.

STREAMS VS TRADITIONAL LOOPS

Streams provide a higher-level, declarative way to process collections compared to imperative, verbose loops.

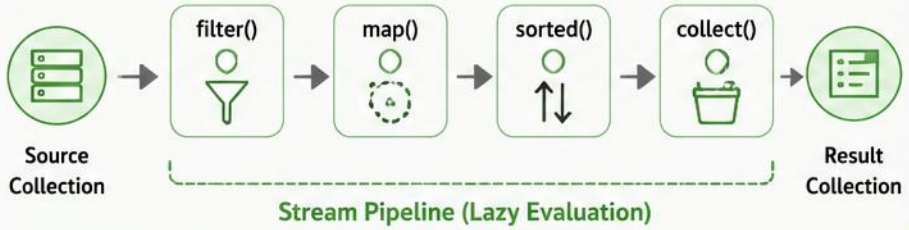
- Loops give you control over every step (iteration, state management).
- Streams express what you want, not how to do it.
- Streams are concise, readable, and less error-prone.
- Streams support functional-style operations and easy parallelism.
- Loops may be more performant in some simple scenarios.
- Choose the right tool based on the use case.

KEY TAKEAWAY Neither approach is universally better — the right choice depends on the problem you're solving.

STREAMS vs TRADITIONAL LOOPS (Java)

Two ways to process collections – same outcome, different approach

JAVA STREAMS (Declarative)

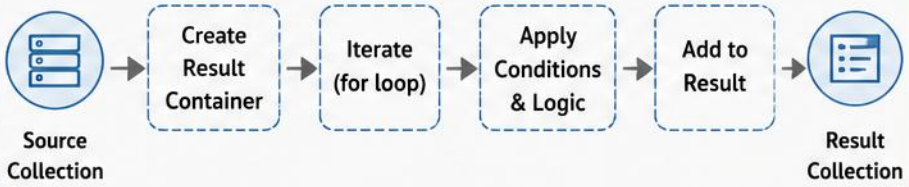


Example: Get names of employees earning > 50000, convert to upper case and sort

```
List<String> result = employees.stream()
            .filter(e -> e.getSalary() > 50000)
            .map(e -> e.getName().toUpperCase())
            .sorted()
            .collect(Collectors.toList());
```

VS

TRADITIONAL LOOPS (Imperative)



Example: Get names of employees earning > 50000, convert to upper case and sort

```
List<String> result = new ArrayList<>();
for (Employee e : employees) {
    if (e.getSalary() > 50000) {
        result.add(e.getName().toUpperCase());
    }
}
Collections.sort(result);
```

Declarative style – What to do, not How	Approach	Imperative style – Step by step How to do
Concise and readable	Code Size	More verbose
Lazy evaluation – operations executed on demand	Evaluation	Eager evaluation – executed immediately
Built-in support for parallel processing	Performance	Manual effort needed for parallelism
Easy to chain multiple operations	Composition	Harder to compose, more boilerplate
Less prone to errors (e.g., forgetting to add)	Safety	More room for errors

Use **Streams** for concise, readable and functional style code. | Use **Traditional Loops** when you need fine-grained control or complex iterations.

EXAMPLE PROBLEM

Find HR employees earning > 60,000, sort by name, and collect into a list.

ID	NAME	DEPARTMENT	SALARY
1	Alice	HR	70000
2	Bob	IT	55000
3	Charlie	HR	65000
4	David	HR	75000
5	Eve	IT	60000
6	Frank	HR	50000

Expected Output: Alice, Charlie, David

KEY TAKEAWAY David qualifies too — salary 75000 in HR — even though he's last alphabetically among the three.

SOLUTION: USING STREAMS

Concise, expressive, and easy to convert to a parallel stream.

```
List<String> result = employees.stream()
    .filter(e -> e.getDepartment().equals("HR"))
    .filter(e -> e.getSalary() > 60000)
    .map(Employee::getName)
    .sorted()
    .collect(Collectors.toList());  // [Alice, Charlie, David]
```

- Why it's better: concise and expressive; focus on what, not how; easy to read and maintain; easy to parallelize.

KEY TAKEAWAY The stream version reads almost like a description of the requirement itself.

SOLUTION: USING TRADITIONAL LOOPS

Same result, more code, and manual state management.

```
List<String> result = new ArrayList<>();
for (Employee e : employees) {
    if (e.getDepartment().equals("HR")) {
        if (e.getSalary() > 60000) {
            result.add(e.getName());
        }
    }
}
Collections.sort(result);  // [Alice, Charlie, David]
```

- Considerations: more lines of code, manual state management, more prone to errors, harder to parallelize.

KEY TAKEAWAY Nested if statements and manual sorting are exactly the kind of boilerplate streams eliminate.

DETAILED COMPARISON

Seven dimensions where streams and loops genuinely differ.

ASPECT	STREAMS	TRADITIONAL LOOPS
Programming Style	Declarative (what to do)	Imperative (how to do it)
Code Readability	More concise and expressive	More verbose
Maintainability	Easier to maintain	Harder as complexity grows
Parallelism	Easy with parallelStream()	Manual threading required
State Management	Handled by framework	Manual management
Error Proneness	Less prone	More prone (off-by-one, nulls)
Flexibility	Less flexible for complex logic	More flexible for complex scenarios

KEY TAKEAWAY Streams win on readability and parallelism; loops win on fine-grained control and flexibility.

WHEN TO USE WHAT?

Match the tool to the shape of the problem.

Use Streams When

- Performing transformations, filtering, mapping, sorting, or aggregations.
- You want concise, readable code.
- You may benefit from parallel processing.
- Working with large collections or I/O data.

Use Loops When

- You need fine-grained control over iteration.
- Working with complex business logic.
- You need early exit or custom control flow.
- Performance tuning simple iterations.

KEY TAKEAWAY Prefer streams for data processing; prefer loops for complex control flow.

COMMON OPERATIONS: STREAMS VS LOOPS (1 OF 2)

Four data-shaping tasks, expressed both ways.

TASK	STREAMS	TRADITIONAL LOOPS
`Filter`	filter(predicate)	if condition inside loop
`Map / Transform`	map(mapper)	Apply logic in loop
`Sort`	sorted()	Collections.sort()
`Count`	count()	int count = 0; increment

KEY TAKEAWAY Four more common operations continue on the next slide.

COMMON OPERATIONS: STREAMS VS LOOPS (2 OF 2)

Four aggregation and matching tasks, expressed both ways.

TASK	STREAMS	TRADITIONAL LOOPS
`Sum`	mapToInt(...).sum()	int sum = 0; sum += value;
`Find First`	findFirst()	break when found
`Any Match`	anyMatch(predicate)	boolean flag + break
`Collect to List/Set`	collect(toList()/toSet())	Add to list/set inside loop

KEY TAKEAWAY Streams are lazy and execute at the terminal operation — loops execute immediately, line by line.

PERFORMANCE CONSIDERATIONS

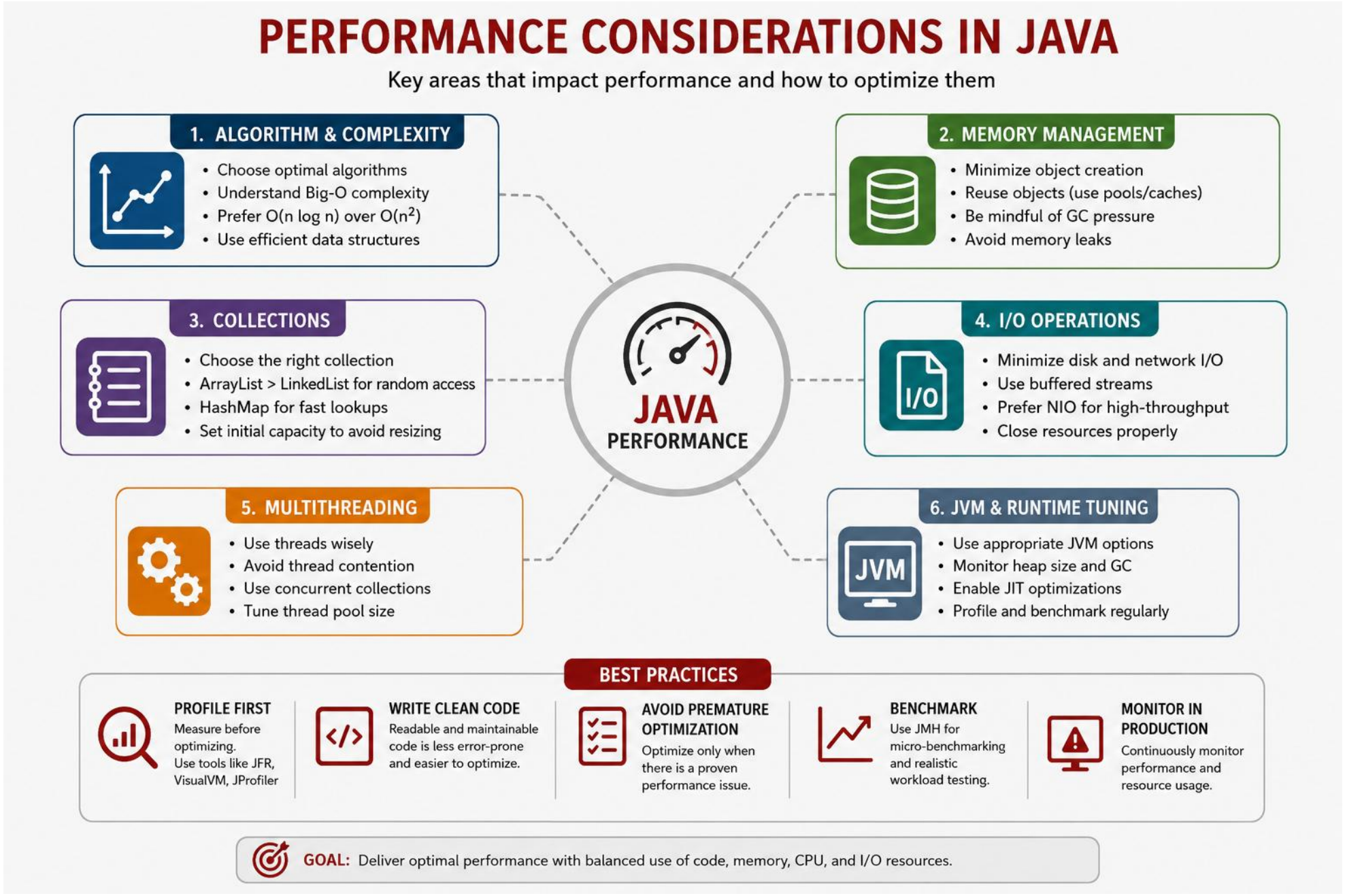
Streams are expressive, but understanding the trade-offs helps you write efficient, scalable applications.

- Performance depends on data size, operation type, and pipeline design.
- Use streams for readability and maintainability, but measure when performance is critical.
- Avoid unnecessary operations and boxing/unboxing.
- Be careful with stateful operations (sorted(), distinct(), limit(), etc.).
- Parallel streams can improve performance for large datasets, but may add overhead for small data.
- Always benchmark and profile before and after optimization.

KEY TAKEAWAY Streams are not always faster — measure, don't guess.

PERFORMANCE CONSIDERATIONS IN JAVA

Key areas that impact performance and how to optimize them



WHEN STREAMS PERFORM WELL — AND WHEN THEY DON'T

Two lists worth memorizing before you reach for a stream by default.

✅ Streams Perform Well When...

- Large datasets benefit from declarative processing.
- Complex transformations, filtering, grouping, aggregations.
- Leveraging parallel processing.
- Readability and maintainability matter.

⚠️ Streams May Not Be Ideal When...

- Very small datasets (stream creation overhead).
- Simple iteration with trivial logic.
- Performance-critical inner loops.
- Fine-grained control over iteration is required.

KEY TAKEAWAY The overhead of building a stream pipeline is real — it just doesn't matter until the dataset is trivially small.

OPERATION COST GUIDE

Not all stream operations cost the same.

OPERATION TYPE	EXAMPLES	COST	NOTES
`Stateless Intermediate`	filter(), map(), peek()	Low	Simple and efficient, can be fused.
`Stateful Intermediate`	sorted(), distinct(), limit(), skip()	Med–High	May require buffering or reordering.
`Short-Circuiting`	anyMatch(), findFirst(), limit()	Low–Med	Can stop early, saves work.
`Terminal (Non-Short-Circuit)`	collect(), forEach(), reduce()	Med–High	Processes all elements.
`Parallel Streams`	parallelStream()	Variable	High overhead for small data; benefits large data.

KEY TAKEAWAY Place cheap, stateless operations like filter() before expensive ones like sorted() to reduce work early.

STREAMS VS LOOPS: A REAL EXAMPLE

Sum of squares of even numbers, 0 to 1,000,000 — both approaches, same result.

USING STREAMS

```
long sum = IntStream.range(0, 1_000_000)
    .filter(i -> i % 2 == 0)
    .mapToLong(i -> (long) i * i)
    .sum();
```

USING A TRADITIONAL LOOP

```
long sum = 0L;
for (int i = 0; i < 1_000_000; i++) {
    if (i % 2 == 0) sum += (long) i * i;
}
```

- Parallel streams — Rule of Thumb: use only for large datasets (10,000+ elements) with CPU-intensive operations.
- Costs: thread management overhead, order may not be preserved, not beneficial for I/O-bound tasks.

KEY TAKEAWAY Both produce the same result — performance varies with JVM optimizations, data size, and pipeline complexity.

SAMPLE BENCHMARK & TUNING CHECKLIST

JMH results for 1 million integers — parallel only wins at scale.

APPROACH	ENVIRONMENT	AVG TIME (MS)	NOTES
`Traditional Loop`	Single Thread	28.7	Baseline
`Stream (Sequential)`	Single Thread	34.2	~19% slower (overhead)
`Stream (Parallel)`	8 cores	9.1	~3× faster for large data
`Stream (Parallel)`	8 cores, 1K data	35.8	Overhead dominates

- Is the dataset large enough to justify streams or parallel streams?
- Can I use primitive streams (IntStream, LongStream) to avoid boxing?
- Are expensive operations placed after filters? Can a short-circuiting terminal operation be used?
- Have I removed unnecessary intermediate operations, and benchmarked with representative data?

KEY TAKEAWAY Parallel streams only pay off at scale — at 1,000 elements the overhead dominates and it's actually slower.

BEST PRACTICES & COMMON PITFALLS

What to do, and the mistakes that catch even experienced developers.

✅ Best Practices

- Avoid unnecessary boxing/unboxing — use primitive streams.
- Place filter() before expensive operations.
- Use limit(), findFirst(), anyMatch() to short-circuit.
- Measure performance with JMH, VisualVM, JProfiler.

⚠️ Common Pitfalls

- Using streams for simple loops adds overhead without benefit.
- Using parallel streams for small datasets.
- Ignoring the cost of stateful operations like sorted().
- Assuming streams are always faster without measuring.

KEY TAKEAWAY Understand the trade-offs, design your pipeline wisely, and measure performance to make the right choice.

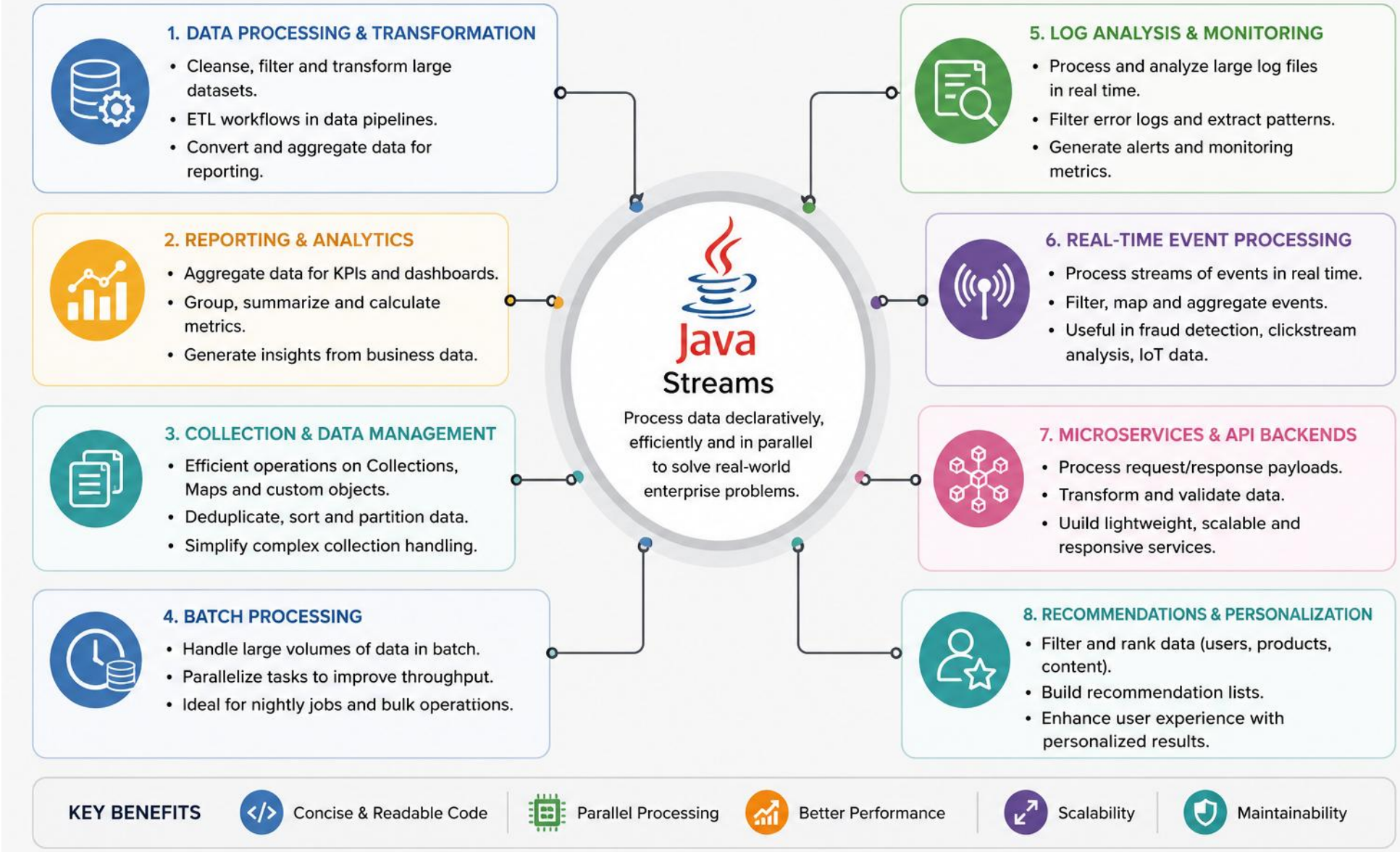
ENTERPRISE USE CASES FOR STREAMS

Streams simplify complex data processing and help build efficient, scalable enterprise applications.

- Ideal for collections, large datasets, and real-time data flows.
- Improve productivity with concise, expressive code.
- Easily support filtering, transformation, aggregation, grouping, and summarization.
- Enable parallel processing for performance at scale.
- Help build maintainable, testable, and robust applications.

KEY TAKEAWAY Streams give enterprise teams a uniform API for very different data-processing needs.

ENTERPRISE USE CASES FOR STREAMS IN JAVA



WHY STREAMS IN ENTERPRISE?

Five reasons enterprise teams standardize on the Streams API.

Higher Productivity

Less boilerplate, more focus on business logic.

Better Maintainability

Readable, modular, easy to refactor.

Performance at Scale

Lazy evaluation and parallel streams improve throughput.

Versatility

Works with collections, databases (JPA), files, real-time data.

Consistency

Uniform API for diverse data-processing needs.

KEY TAKEAWAY Consistency is underrated — the same filter/map/collect vocabulary applies everywhere in the codebase.

ENTERPRISE USE CASES — PART 1

Filtering, transformation, aggregation, and grouping.

USE CASE	DESCRIPTION	EXAMPLE OPERATIONS	BUSINESS VALUE
Data Filtering & Validation	Filter records based on business rules and quality checks	filter(), allMatch(), anyMatch()	Accurate, high-quality data
Data Transformation	Transform data into required formats or structures	map(), flatMap()	Standardizes data for processing
Aggregation & Summarization	Totals, averages, counts, min/max, summaries	collect(), reduce(), summarizingInt()	Business insights and KPIs
Grouping & Categorization	Group data by attributes, aggregate per group	groupingBy(), partitioningBy()	Helps reporting and segmentation

KEY TAKEAWAY Filtering and transformation are the entry point for almost every enterprise data pipeline.

ENTERPRISE USE CASES — PART 2

Joining, deduplication, reporting, and real-time processing.

USE CASE	DESCRIPTION	EXAMPLE OPERATIONS	BUSINESS VALUE
Data Joining & Merging	Combine data from multiple sources or collections	flatMap(), joining(), toMap()	Unified views across systems
Deduplication	Remove duplicates, ensure unique records	distinct(), toSet()	Data integrity, less redundancy
Reporting & Dashboards	Generate reports with aggregated data	groupingBy(), counting(), summing()	Real-time analytics for decisions
Real-time Event Processing	Process streaming events from queues, logs, sensors	filter(), map(), windowing	Low-latency insights and alerting

KEY TAKEAWAY Real-time event processing is where streams and message queues (Kafka, RabbitMQ) meet in production systems.

CODE EXAMPLE: EMPLOYEE ANALYTICS

Average salary per department, for employees earning more than 60,000.

```
Map<String, Double> avgSalaryByDept = employees.stream()
    .filter(e -> e.getSalary() > 60000)
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)));
avgSalaryByDept.forEach((dept, avg) ->
    System.out.println(dept + " : " + avg));
```

DEPT	AVG SALARY
HR	74250.0
IT	68500.0
Finance	91200.0

KEY TAKEAWAY groupingBy() combined with a downstream collector is the pattern behind most enterprise analytics queries.

STREAMS IN ENTERPRISE SYSTEMS

Where streams show up when processing real production data.

- Databases (query processing) — Files & Logs (ETL, parsing, analytics)
- Message Queues (Kafka, RabbitMQ, ActiveMQ) — APIs & Services (transform/aggregate responses)
- Reports & Dashboards (analytics and visualizations)

👍 Parallel Benefits

- Utilizes multiple processor cores.
- Faster processing for large datasets.

⚠️ Considerations

- Not beneficial for small datasets.
- Order may not be preserved.
- Avoid shared mutable state.

KEY TAKEAWAY parallelStream() is best suited for large datasets and computationally intensive workloads.

PARALLEL STREAM EXAMPLE & BEST PRACTICES

Filtering high-value orders across many cores.

```
List<Customer> highValueCustomers = orders.parallelStream()
    .filter(o -> o.getAmount() > 10000)
    .map(Order::getCustomer)
    .distinct()
    .collect(Collectors.toList());
```

- Prefer streams for complex data processing; use meaningful method references and lambdas.
- Avoid side effects in intermediate operations; always handle null values appropriately.
- Consider parallel streams only after profiling and testing; measure performance in production.

KEY TAKEAWAY Choose streams for clarity and productivity; choose parallel streams for scale and speed.

TRY IT YOURSELF — EXERCISE 8: PARALLELSTREAM BONUS

Reinforces: the parallel stream example you just saw.

STEP	CODE	RESULT
Goal	Compare count() using stream() vs. parallelStream()	Both counts match
Code	.stream().count() vs .parallelStream().count()	Same result either way
Key concept	On tiny data, parallel overhead can outweigh any benefit	Ties to the parallel stream example you just saw
Reference	exercises/exercise-08-parallel-bonus.md	Full starter code and steps

TRY IT NOW Complete Exercise 8 before continuing — see exercises/exercise-08-parallel-bonus.md.

PRE-LAB EXERCISES OVERVIEW

Complete these 8 short exercises before Lab 6 — practice Streams and Lambdas on employee data first.

Exercise Objectives

- Use lambdas to write concise code.
- Build pipelines to filter, transform, aggregate.
- Apply common stream operations.
- Compare streams with traditional loops.

Prerequisites

- Java 8 or higher.
- Basic knowledge of Collections.
- IDE (IntelliJ / Eclipse / VS Code).
- Understanding of lambdas (Module 5).

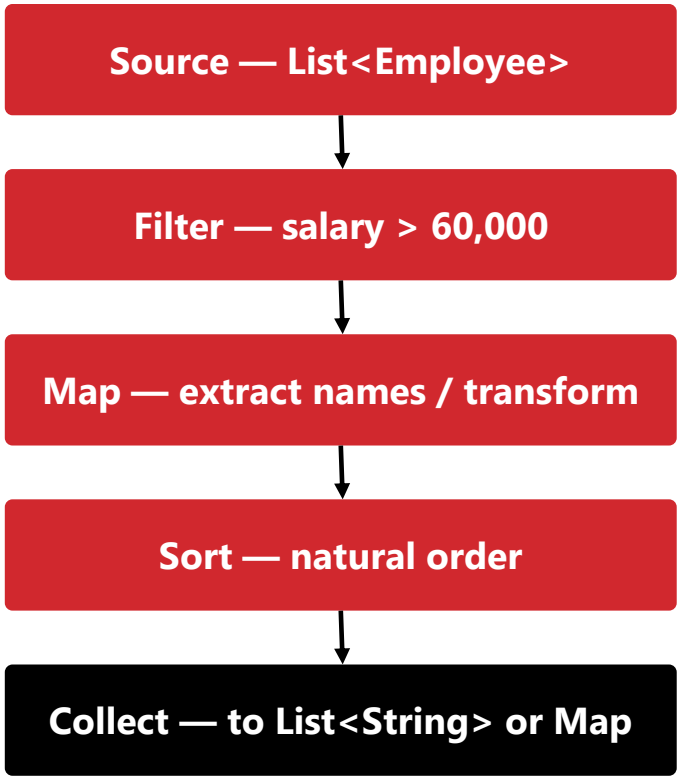
Scenario

- Given a list of employees.
- Perform operations using Streams and Lambdas.
- Answer business questions.

KEY TAKEAWAY Dataset is provided as a List<Employee> in the starter code — no data setup required.

PRE-LAB EXERCISE PIPELINE SHAPE

Filter, map, sort, collect — the same shape you will reuse throughout Lab 6.



KEY TAKEAWAY Terminal operations (collect(), forEach(), count()) are what actually trigger execution.

PRE-LAB EXERCISES: THE 8 TASKS

Seven core exercises, plus a bonus comparing sequential and parallel streams.

- Declare a custom functional interface and implement it with an anonymous class and a lambda.
- List employees with salary greater than 60,000.
- List all employee names.
- Find the highest and lowest salary.
- Increase every employee's salary by 10% and return a list of updated salaries.
- Count employees in each department.
- Get the names of employees working in the HR department.
- Bonus: use parallelStream() to count employees and compare performance with stream().

KEY TAKEAWAY Each exercise maps directly to a stream operation covered earlier in this module.

PRE-LAB EXERCISES: CODE STARTER (SAMPLE)

The Employee class and dataset provided for the pre-lab exercises.

```
class Employee {
    int id; String name; String department; double salary; LocalDate joinDate;
    Employee(int id, String name, String dept, double salary, LocalDate joinDate) { ... }
    // getters and toString()
}

List<Employee> employees = Arrays.asList(
    new Employee(101, "Alice", "HR", 72000.0, LocalDate.of(2022, 5, 10)),
    new Employee(102, "Bob", "IT", 65000.0, LocalDate.of(2021, 3, 15)),
    new Employee(103, "Charlie", "HR", 80000.0, LocalDate.of(2020, 7, 20)),
    new Employee(104, "Diana", "Finance", 90000.0, LocalDate.of(2019, 11, 5)),
    new Employee(105, "Evan", "IT", 55000.0, LocalDate.of(2023, 1, 12))
);
```

KEY TAKEAWAY This exact 5-employee dataset drives every expected output in the pre-lab exercises — Lab 6 later uses a larger 25-employee roster.

PRE-LAB EXERCISES: EXPECTED OUTPUT (SAMPLE)

What each of the seven exercises should produce, given the starter dataset.

```
1. Anonymous: true   Lambda: true
2. [Alice, Bob, Charlie, Diana]
3. [Alice, Bob, Charlie, Diana, Evan]
4. Highest Salary: 90000.0 (Diana)   Lowest Salary: 55000.0 (Evan)
5. [79200.0, 71500.0, 88000.0, 99000.0, 60500.0]
6. {HR=2, IT=2, Finance=1}
7. [Alice, Charlie]
```

Useful operations: filter(), map(), sorted(), collect(), groupingBy(), max()/min(), Comparator.comparing()

KEY TAKEAWAY Actual output may vary if the dataset changes — but the operations you'd use stay the same.

PRE-LAB EXERCISES: TIPS, HINTS & EXTENSIONS

Guidance for the pre-lab exercises, plus optional stretch goals.

💡 Tips & Hints

- Use method references (Employee::getName) for cleaner code.
- Use Collectors.groupingBy() for grouping/aggregation.
- Intermediate ops are lazy; terminal ops execute the pipeline.
- Prefer immutable results — avoid modifying source data.

🚀 Extensions (Optional)

- Group employees by department and calculate average salary.
- Find the top 3 highest-paid employees.
- Partition into high earners (>70,000) and others.
- Generate a report string for each department.

KEY TAKEAWAY Finish these exercises, then move on to Lab 6 — the full Employee Analytics System.

LAB 6 OVERVIEW — EMPLOYEE ANALYTICS SYSTEM

Build a menu-driven console app that turns a 25-employee roster into management reports using Streams and Lambdas.

Lab 6 Objectives

- Use lambdas and core functional interfaces (Predicate, Function, Consumer, Supplier).
- Build stream pipelines: filter, map, sorted, distinct, limit, skip.
- Aggregate with reduce(), collectors, groupingBy(), and partitioningBy().
- Handle missing values safely with Optional and build an analytics Dashboard.

Prerequisites

- JDK 21 and IntelliJ IDEA Community (or VS Code).
- Lab 0 environment setup completed.
- Comfort with List, packages, and Lab 5 service-layer patterns.
- Module 6 pre-lab exercises completed (see previous slides).

KEY TAKEAWAY Lab 6 is the real hands-on capstone for Module 6 — a full Employee Analytics System, not just the pre-lab exercises.

LAB 6 DATASET — EMPLOYEE ANALYTICS

The Employee model used throughout Lab 6 — 25 sample employees across five departments.

FIELD	TYPE	DESCRIPTION	EXAMPLE
`employeeId`	String	Primary employee key	E001
`name`	String	Employee name	John Smith
`department`	String	IT, HR, Finance, Sales, Marketing	IT
`salary`	double	Annual salary (48,000–165,000)	165000
`active`	boolean	Employment status	true

KEY TAKEAWAY Employee also has experience and rating fields — seven in total feed the Dashboard and every report.

LAB 6 CORE TASKS — MENU-DRIVEN CONSOLE

Nine graded menu options (1–9) — build them in order before any bonus features.

- 1. Display Employees — print the full 25-employee roster via forEach().
- 2. Employees By Department — group with Collectors.groupingBy(Employee::getDepartment).
- 3. Salary Report — reduce(), summarizingDouble(), and partitioningBy() by salary threshold.
- 4. Top Performers — filter and sort employees with rating >= 4.
- 5. Highest Salary — max(Comparator.comparingDouble(...)) returned as Optional<Employee>.
- 6. Department Statistics — groupingBy() + summarizingDouble() per department.
- 7. Active Employees — filter(Employee::isActive).
- 8. Dashboard — headcount, salary stats, top performer, top 5 salaries, active/inactive counts.
- 9. Exit — clean shutdown of the Scanner and menu loop.

KEY TAKEAWAY Complete CORE menu options 1–9 first; bonus features (menu 10+) come only after CORE works.

LAB 6 — KEY STREAM OPERATIONS USED

Five categories of operations power every report in the Employee Analytics System.

CATEGORY	OPERATION	EXAMPLE
Functional Interfaces	Predicate, Function, Consumer, Supplier	Predicate<Employee> highEarner = e -> e.getSalary() > 100_000
Filtering & Mapping	filter(), map(), method references	filter(Employee::isActive).map(Employee::getName)
Sorting & Slicing	sorted(), distinct(), limit(), skip()	sorted(Comparator.comparingDouble(Employee::getSalary).reversed()).limit(5)
Reduction	reduce(), mapToDouble().sum()/average()	mapToDouble(Employee::getSalary).average()
Collectors & Optional	groupingBy(), partitioningBy(), summarizingDouble()	groupingBy(Employee::getDepartment, summarizingDouble(Employee::getSalary))

KEY TAKEAWAY Every menu option composes these same five categories — that judgment, not the syntax, is what transfers to CRM reporting later.

LAB 6 — EXPECTED DELIVERABLES

Six things to submit once CORE menu options 1–9 and the Dashboard are working.

- 1. Complete Java Project — Employee, EmployeeData, EmployeeService, ReportService, Main under com.academy.analytics.
- 2. Screenshots — the running menu, filters/groups, and the Dashboard (option 8).
- 3. Stream Operations Table — which operations you implemented and where.
- 4. README / LMS Notes — overview, operations used, functional interfaces, how to compile and run.
- 5. Reflection Answers — written in notes/lab6-answers.md.
- 6. Bonus (Optional) — labeled stretch challenges from menu options 10+.

KEY TAKEAWAY Graders recompile your sources with javac -d out — the Dashboard screenshot is the single most important piece of evidence.

LAB 6 — EVALUATION RUBRIC & DASHBOARD OUTPUT

How the 100-mark rubric is graded, and what the Dashboard (menu option 8) should print.

- Graded out of 100: Project Structure (10), Lambda Expressions (10), Functional Interfaces (10), Stream Pipeline (15).
- Filtering/Mapping/Sorting (15), Collectors & Grouping (15), Reduction & Statistics (10), Dashboard & Reports (10), Code Quality (5).

```
Employees : 25   Average Salary : 99720   Highest Salary : 165000
Departments : 5   Top Performer : John Smith (Rating 5)
Highest Paid Department : IT
Active Employees : 23   Inactive Employees : 2
```

KEY TAKEAWAY Stream Pipeline, Filtering/Mapping/Sorting, and Collectors & Grouping carry the most marks — 45 of the 100.

LAB 6 — TIPS & REFLECTION

Common pitfalls to avoid, and the reflection questions that close out the lab.

- A Stream can be consumed only once — call .stream() again for each new query instead of reusing a variable.
- Never call .get() on an Optional — use ifPresent() / ifPresentOrElse() / orElse() instead.
- Apply skip() after the same sort used for limit() — sorting after skipping produces nonsense pages.
- Recompile with javac -d out after every change — stale .class files hide your fixes.

Reflect on: why streams postpone work until a terminal operation, and how groupingBy() will reuse for CRM customer reports later.

KEY TAKEAWAY Eleven reflection questions in notes/lab6-answers.md turn a working Dashboard into real understanding of Streams.

MODULE 6 SUMMARY

Java Streams and Lambda expressions provide a powerful, declarative, and concise way to process data.

- Streams enable functional-style programming using lambda expressions.
- Pipelines are built with intermediate operations and executed by terminal operations.
- Operations are lazy, concise, and promote readability.
- Streams support powerful data processing on collections, arrays, files, and more.
- Use Streams for maintainable, testable, and scalable enterprise applications.

KEY TAKEAWAY This module covered the core concepts, operations, and practical usage of Streams through hands-on labs.

CORE CONCEPTS COVERED

Six pillars this module built up, one at a time.

λ Lambda Expressions

- Provide concise, anonymous functions.

🔗 Stream Pipelines

- Source → Intermediate Operations → Terminal Operation.

⏳ Lazy Evaluation

- Intermediate operations execute only when needed.

🧰 Stream Operations

- Filtering, mapping, sorting, aggregation, collecting.

📦 Collectors Framework

- Flexible ways to collect and summarize results.

⚖️ Performance

- When streams help, and when loops are more appropriate.

KEY TAKEAWAY Every later topic in this module built directly on lambda expressions and the pipeline shape.

BENEFITS OF STREAMS

Five reasons streams have become the default for data processing in modern Java.

Concise & Expressive

Less boilerplate, more focus on business logic.

Readability & Maintainability

Clear processing pipelines improve understanding.

Performance at Scale

Supports parallel processing for large datasets.

Flexibility

Works with many data sources, complex operations.

Consistency

Uniform API for diverse data-processing tasks.

KEY TAKEAWAY Concise code and consistent APIs compound over the life of a codebase — small savings, repeated everywhere.

WHEN TO USE — AND AVOID — STREAMS

The decision comes down to data size, complexity, and control needs.

✅ Use Streams When You Need...

- Complex data transformations.
- Filtering, mapping, and aggregation.
- Processing large datasets.
- Parallel processing for performance.
- Improved readability and maintainability.

🚫 Consider Loops When...

- Very simple iterations (stream overhead unnecessary).
- Performance-critical tight loops.
- Small datasets where readability gains are minimal.

KEY TAKEAWAY Loops still have their place when you need full control and flexibility.

IMPORTANT OPERATIONS REFERENCE (1 OF 2)

Four intermediate operations, and what each one does.

CATEGORY	COMMON OPERATIONS	PURPOSE
Filtering	filter()	Select elements matching a condition
Mapping	map()	Transform each element
Sorting	sorted()	Sort stream elements
Aggregation	collect(), reduce(), summarizing*()	Aggregate or summarize data

KEY TAKEAWAY Four more categories — grouping through collecting — continue on the next slide.

IMPORTANT OPERATIONS REFERENCE (2 OF 2)

Four more categories, and what each one does.

CATEGORY	COMMON OPERATIONS	PURPOSE
`Grouping`	groupingBy()	Group elements by key
`Counting`	count()	Count elements
`Finding`	findFirst(), anyMatch(), allMatch()	Search or match elements
`Collecting`	toList(), toSet(), toMap()	Collect into data structures

KEY TAKEAWAY Bookmark this table — it's the fastest lookup for choosing the right operation under time pressure.

LAB HIGHLIGHTS & TOOLS

What you actually did during the hands-on lab, and what you did it with.

During the Lab, You...

- Worked with a real-world employee dataset.
- Applied filtering, mapping, sorting, aggregation.
- Used Collectors to summarize and group data.
- Compared the Stream approach with traditional loops.
- Generated useful business insights.

Tools & Technologies

- Java 8+
- IntelliJ IDEA / Eclipse / VS Code
- Java Collections Framework
- Java Streams API
- Lambda Expressions

KEY TAKEAWAY Built maintainable and scalable solutions — that's the real goal behind every task in the lab.

WHAT YOU CAN DO NOW

The skills this module leaves you with.

- Write clean and concise data-processing code using Streams and Lambdas.
- Build efficient pipelines for transformation, filtering, and aggregation.
- Use Collectors to create meaningful summaries and reports.
- Decide when to use sequential versus parallel streams.

"Streams help you express the 'what', not the 'how'. Write less code. Achieve more."

KEY TAKEAWAY Intermediate operations are lazy; terminal operations trigger execution — choose the right operation for the job.

KNOWLEDGE CHECK — MULTIPLE CHOICE (1–6)

Choose the best answer. Correct answers are marked ✓.

1. Which of the following best describes a Stream?

A) A data structure that stores elements **B) A sequence of elements supporting functional-style operations ✓** C) A class used to read data from files D) A thread used for parallel processing

2. Which operation type does not trigger execution of the stream pipeline?

A) filter() B) map() **C) sorted() ✓** D) collect()

3. What is the purpose of the map() operation?

A) To filter elements based on a condition **B) To transform each element ✓** C) To sort elements D) To remove duplicates

4. Which terminal operation returns the number of elements in a stream?

A) count() ✓ B) forEach() C) collect() D) reduce()

5. How does lazy evaluation benefit stream pipelines?

A) It improves code readability **B) It avoids unnecessary computation ✓** C) It increases memory usage D) It executes operations in parallel

6. Which method processes each element without returning a result?

A) map() B) filter() **C) forEach() ✓** D) reduce()

KEY TAKEAWAY Six more multiple-choice questions continue on the next slide.

KNOWLEDGE CHECK — MULTIPLE CHOICE (7–12)

Choose the best answer. Correct answers are marked ✓.

7. Which collector would you use to convert a stream to a List?

A) Collectors.toSet() **B) Collectors.toList() ✓** C) Collectors.toMap() D) Collectors.toCollection()

8. What is the correct syntax to create a stream from a List<Employee> named emps?

A) Stream<Employee> s = new Stream<>(emps); **B) Stream<Employee> s = emps.stream(); ✓** C) Stream<Employee> s = Stream.of(emps); D) Stream<Employee> s = emps.toStream();

9. Which operation is used to sort elements in natural order?

A) sorted() ✓ B) sort() C) order() D) arrange()

10. Which of the following is true about parallel streams?

A) They always improve performance. B) They maintain encounter order by default. **C) They are created using parallelStream(). ✓** D) They can be used only with primitive types.

11. Which operation combines the elements of a stream into a single value?

A) collect() **B) reduce() ✓** C) map() D) filter()

12. Which statement is true about Lambda expressions?

A) They can modify only instance variables. B) They must have a return statement. **C) They must capture only final or effectively final variables. ✓** D) They can throw checked exceptions only.

KEY TAKEAWAY 12 questions, testing the full span of this module — from stream basics to lambda variable capture.

SELF-ASSESSMENT & BONUS CHALLENGE

How confident are you with Module 6 concepts?

Confident

Ready to apply these concepts independently.

Average

Comfortable with the basics, need more practice.

Need Review

Revisit the pipeline and lazy evaluation sections.

BONUS CHALLENGE

Using List<Employee> employees: filter salary > 60,000, sort by salary descending, get the top 3, collect their names into a List<String>, and print the result.

```
Hint: filter() → sorted() → limit() → map() → collect()
```

KEY TAKEAWAY If you can chain filter → sorted → limit → map → collect from memory, you've mastered this module.

DID YOU KNOW? & REMEMBER

A closing fact, and the four ideas to carry forward.

DID YOU KNOW?

The Streams API was introduced in Java 8 as part of the functional programming enhancements. It seamlessly integrates with collections and offers powerful capabilities for modern Java development.

- Build the pipeline; don't execute early.
- Choose the right operations.
- Terminal operations trigger execution.
- Measure and optimize when needed.

KEY TAKEAWAY Popular entry points: Collection<E>.stream(), Collection<E>.parallelStream(), Arrays.stream(array), Stream.of(values...)

Nicely done

You can write concise, declarative pipelines with Streams and Lambdas instead of imperative loops.

Exception handling and robust error management are next in your toolkit.